Kapitel 2

Vorgehensmodelle der Softwareentwicklung

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Agenda

Abschnitt 2.1
Klassische Vorgehensmodelle

Abschnitt 2.2
V-Modell

Abschnitt 2.3
Agile Vorgehensmodelle

Abschnitt 2.4
Scrum

Abschnitt 2.5
Continuous Integration & DevOps

Abschnitt 2.6
Versionsverwaltung & Rückverfolgbarkeit

Klausurrelevante Lernziele

- Wasserfallmodell erläutern, Vor- und Nachteile nennen können
- Unterschied iterative & inkrementelle Softwareentwicklung erläutern können
- Werte des agilen Manifests nennen und erläutern können
- Unterschied klassisches & agiles Projektmanagement erläutern können
- Aufgabe des Refactoring nennen können
- Test-Driven Development erläutern, Vor- und Nachteile nennen können
- Beispiele nennen können, wann Agilität sinnvoll und wann sie es nicht ist
- Rollen, Artefakte, Ereignisse und prinzipiellen Ablauf von Scrum erläutern können
- Continuous Integration erläutern, einige Grundsätze nennen können
- Ziele der Versionsverwaltung nennen, Unterschied zentrale & verteilte Versionsverwaltung erläutern können
- Bedeutung und Probleme von Branching & Merging erläutern können
- Rückverfolgbarkeit erläutern, Gründe für den Einsatz nennen können

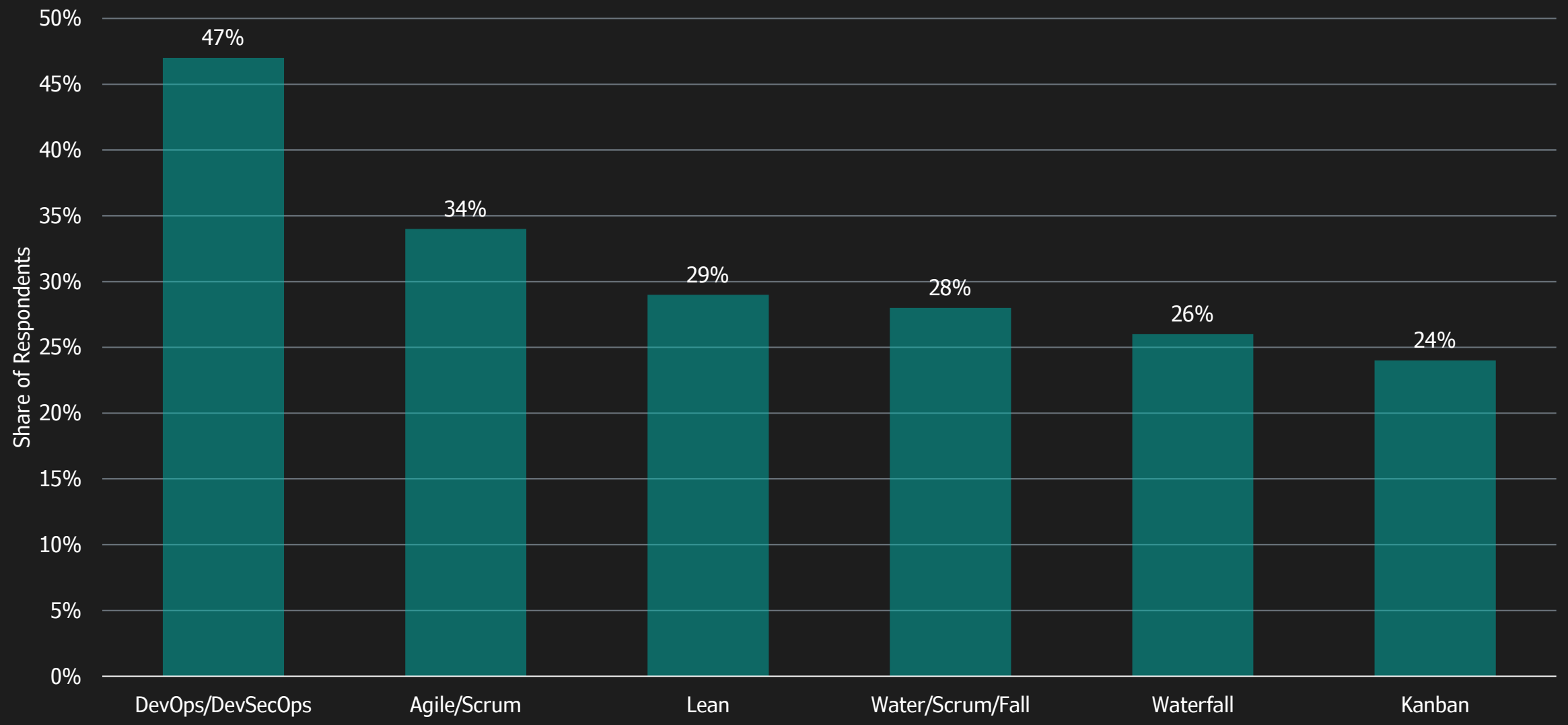
Option: Zertifizierung

- Verschiedene Anbieter, z.B. Scrum.org und Scrum Allicance
- Weltweit anerkannte Zertifikate für die agile Softwareentwicklung mit Scrum
- Professional Scrum Developer I (PSD I)
  - Multiple Choice, 80 Fragen, 60 Minuten, Bestehensquote: 85%
  - Nur englisch, Gebühr: $200
  - Nur Online-Prüfung, Online-Beispielprüfung vorhanden
- Weitere
  - Professional Scrum Product Owner (PSPO)
  - Professional Scrum Master (PSM)
  - Professional Scrum with Kanban (PSK)
  - Professional Agile Leadership (PAL)
  - Scaled Professional Scrum (SPS)
- Vorlesung bereitet weitgehend auf die Prüfung vor

Vorgehensmodelle der Softwareentwicklung

- Strukturieren das allgemeine Vorgehen: sie bilden die Grundlage für die Planung und Durchführung der Softwareentwicklung
  - Achtung: Begriff »Projekt« ist problematisch bei der Softwareentwicklung
- Beschreiben die Aufbauorganisation
  - Teamstruktur, Rollen, Verantwortlichkeiten etc.
- Beschreiben die Ablauforganisation
  - Phasen, Aktivitäten, Ergebnisse, Prozesse etc.
- Beschreiben ggf. einzusetzende Techniken, Methoden, Artefakte, Dokumentation, Werkzeuge, Indikatoren und Metriken etc.

Vorgehensmodelle: Weitweiter Einsatz



Quelle: (GitLab 2022)

Abschnitt 2.1

Klassische Vorgehensmodelle

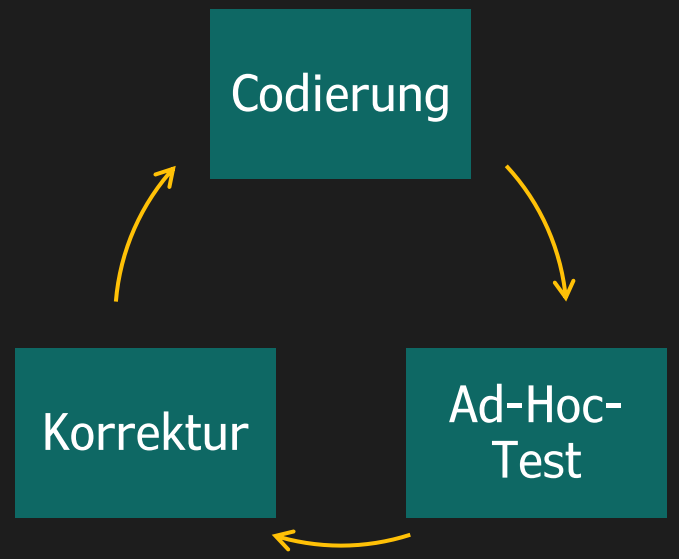
FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Klassische Vorgehensmodelle: Beispiele

- Code & Fix
- Lineare (aka phasenorientierte) Modelle
  - Wasserfallmodell
- Nicht-lineare Modelle
  - Evolutionäre Entwicklung
  - Iterative Entwicklung
  - Inkrementelle Entwicklung
  - Treppenmodell
  - Spiralmodell
- V-Modell XT
- Rational Unified Process (RUP)

Code & Fix

- Einfachstes »Modell«



- Schnell: keine Anforderungen, Spezifikation oder Entwurf; Planung nicht nötig
- Nachteile: verteilte Entwicklung kaum möglich; schlechte Strukturierung; hoher Korrekturaufwand, da Mängel oft erst im Betrieb auffallen
- Gründe: Kompetenz fehlt oder falsche Vorstellung des Managements, z.B.
  - »Software« ist gleich »Code« und Qualitätssicherung nur »Bug Fixing«
  - Probleme bei Software entstehen nur, weil Entwickelnde versagen

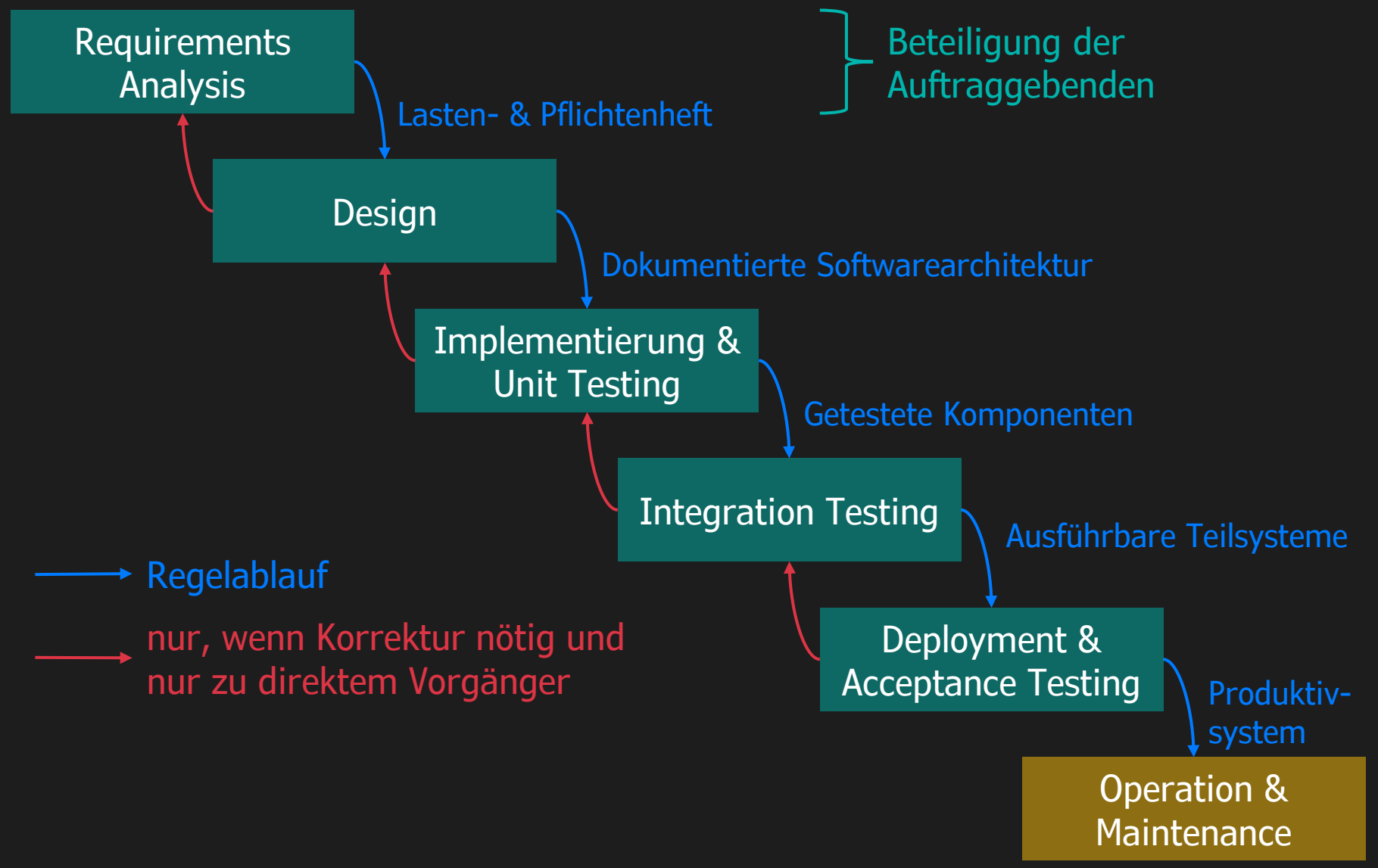
Wasserfallmodell (1)

Waterfall Model – Model of the software development process in which the constituent activities, typically a concept phase, requirements phase, design phase, implementation phase, test phase, and installation and checkout phase, are performed in that order, possibly with overlap but with little or no iteration.

(ISO/IEC/IEEE 24765:2017)

- Lineare Abfolge von Tätigkeiten
- Rückkopplung zu Vorgängertätigkeit, falls Korrekturen nötig sind
- Dokumentengetriebenes Vorgehen: jede Tätigkeit produziert Dokumente als Ergebnis, Übergabe an nächste Tätigkeit

Wasserfallmodell (2)



Wasserfallmodell (3)

Vorteile

- Einfach & intuitiv
- Weit verbreitet
- Notwendigkeit eines strukturierten Vorgehens
- Dokumente
  - Schriftliches Fixieren von Spezifikationen
  - Vertrag zur Übergabe an Folgetätigkeit
  - Als Quasi-Meilensteine im Projekt leicht kontrollierbar
- Prozessablauf transparent

Nachteile

- Auftraggebende nur am Anfang beteiligt: Anforderungen sind aber oft noch nicht eindeutig
- Tests & Abnahme erst spät, wenn System vollständig
- Kontinuierliche Qualitätssicherung problematisch
- »Tätigkeit« und »Phase« sind nicht klar getrennt
- Projektmanagement: Zyklen problematisch, keine Meilensteine zwischen Tätigkeiten

Iterative Softwareentwicklung

Iterative Development – Software development technique in which requirements definition, design, implementation, and testing occur in an overlapping, iterative (rather than sequential) manner, resulting in incremental completion of the overall software product.

(ISO/IEC/IEEE 24765:2017)

- Zerlege die Entwicklung in Folge von kürzeren, abgeschlossenen Iterationen
  - In jeder Iteration: Analysieren, Design & Architektur, Implementierung, Testen
  - Ziel: jede Iteration behebt Mängel und verbessert das System der vorherigen Iteration
- Resultierende Gesamtsystem wird getestet und erst dann ausgeliefert; es werden keine Zwischenstände ausgeliefert
- Zu Beginn ist das genaue Endprodukt noch nicht vollständig geklärt; es wird erst im Laufe der Entwicklung präzisiert

Inkrementelle Softwareentwicklung

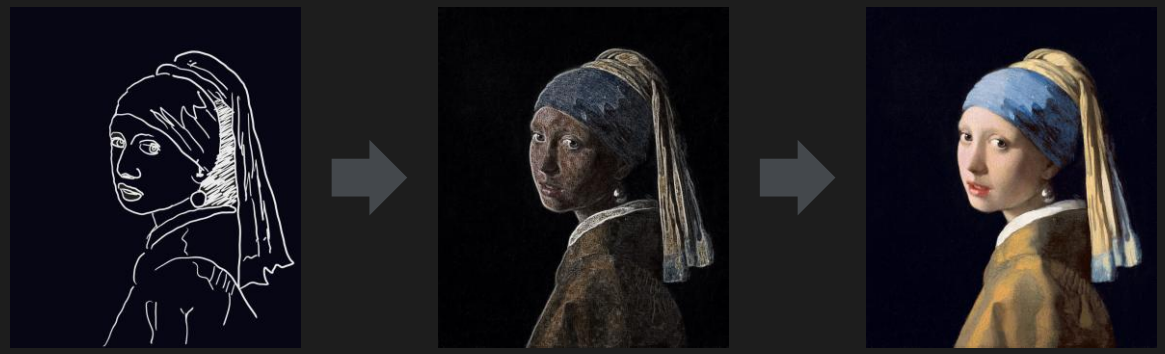
Incremental Development – A type of software development lifecycle model in which the component or system is developed through a series of increments.

(Project Management Institute 2017)

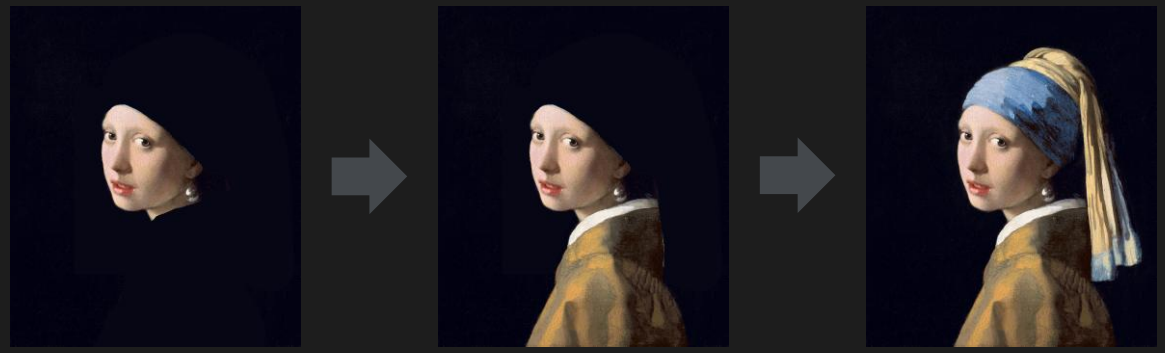
- Endprodukt wird zu Beginn vollständig spezifiziert
- Die Realisierung erfolgt schrittweise durch Inkremente: jedes Inkrement ist lauffähig und wird auch ausgeliefert
- Schon das erste Inkrement (»Kernsystem«) sollte bereits ein funktional nutzbringender Teil des Endprodukts sein
- Vorteil: schon früh lauffähige, wesentliche Teile des Endprodukts

Iterativ, inkrementell & iterativ-inkrementell

Iterativ



Inkrementell



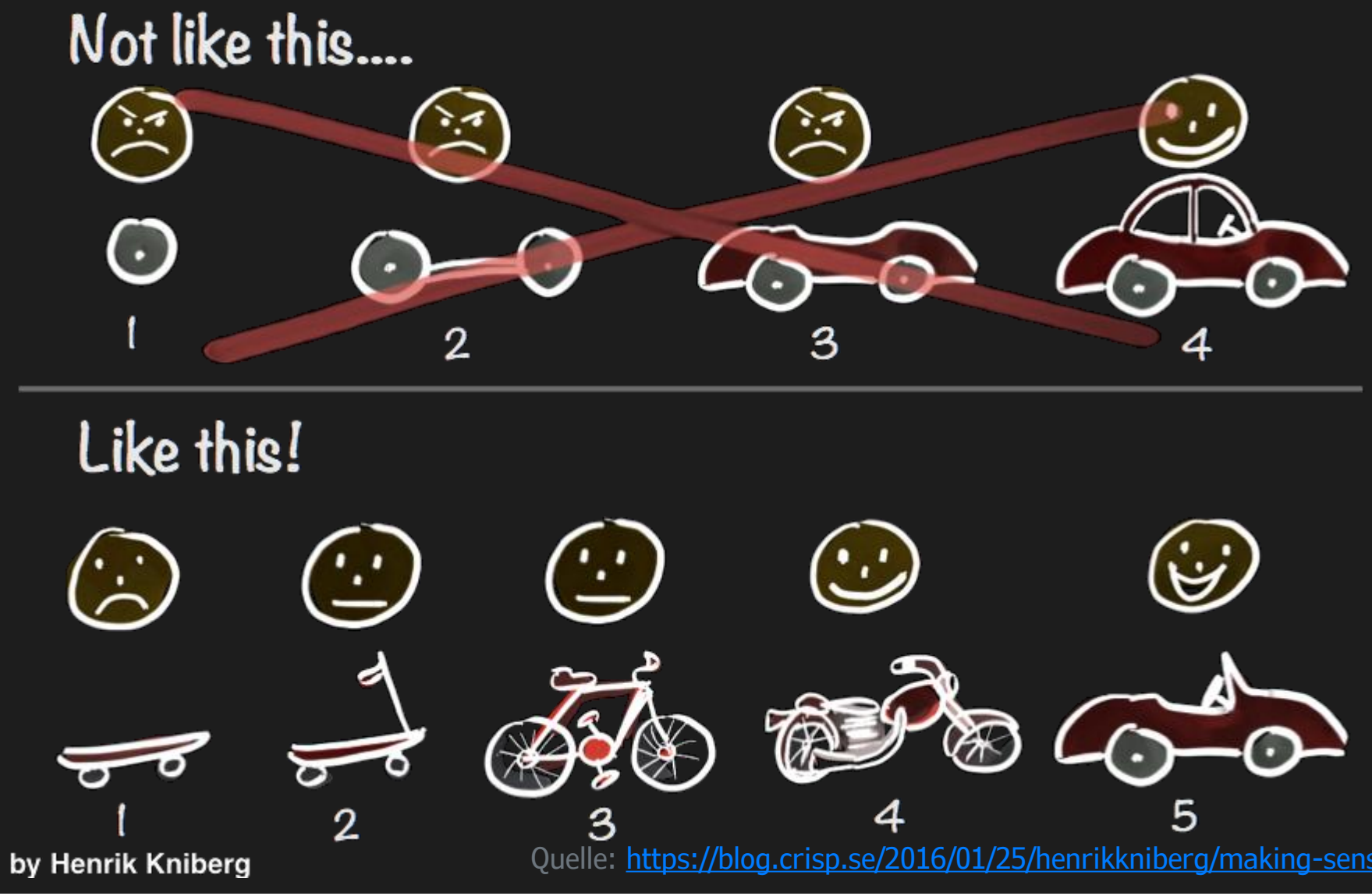
Iterativ-inkrementell

Minimum Viable Product (1)

- Minimum Viable Product (MVP)
  - »Version eines neuen Produkts, mit der ein Entwicklerteam möglichst große Erkenntnisse über Kunden mit möglichst geringem Aufwand gewinnen kann« (Eric Ries, The Lean Startup, 2011)
  - Entwickle MVP → teste mit echten Nutzenden → verbessere Produkt
- Gründe
  - Marktchance mit geringen Entwicklungsaufwand & Risiko prüfen
  - Nachweis Machbarkeit & Kompetenz
  - Frühestmögliche Bereitstellung für Early Adopters
- Verwandte Begriffe: Minimum Viable Team (bei Startups), Minimum Awesome Product (MAP)

Quelle: (Ries 2011)

Minimum Viable Product (2)



Quelle: https://blog.crisp.se/2016/01/25/henrikkniberg/making-sense-of-mvp (2016)

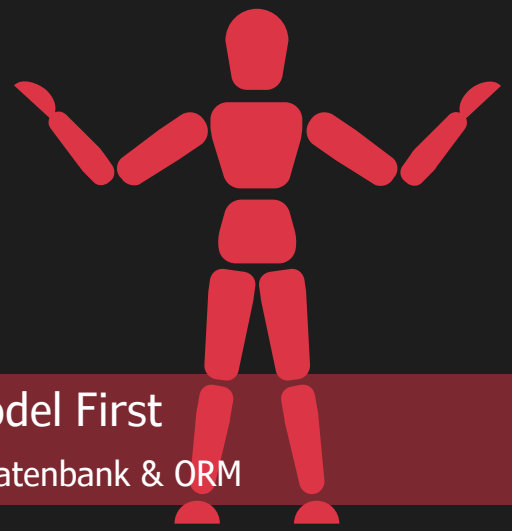
X-First

Mobile First

- Responsive Design

Cloud First

- Hosting & Entwurfsmuster



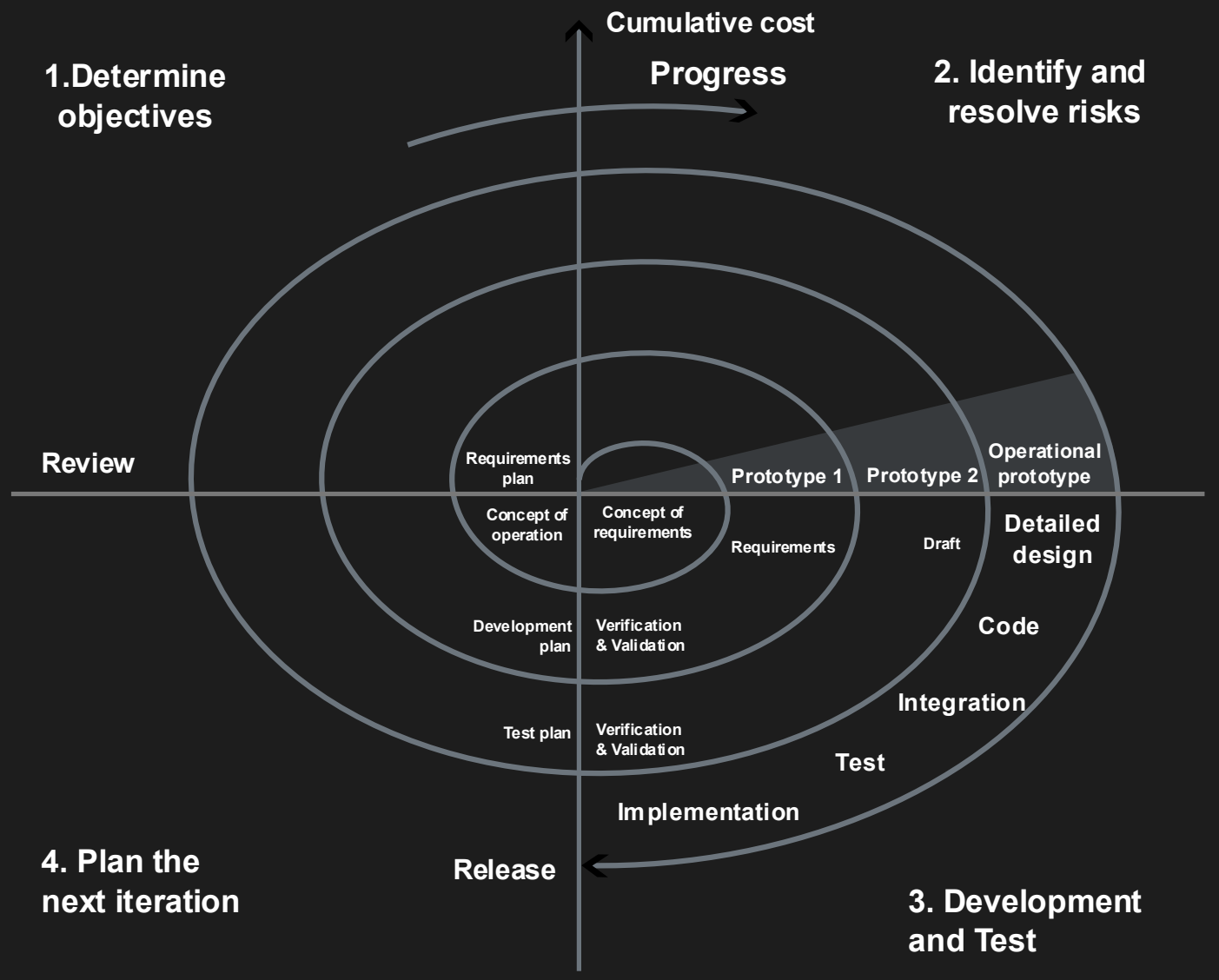
Model First

- Datenbank & ORM

English First

- Lokalisierung & Internationalisierung

Spiralmodell von Boehm



Quelle: (Boehm 1986)

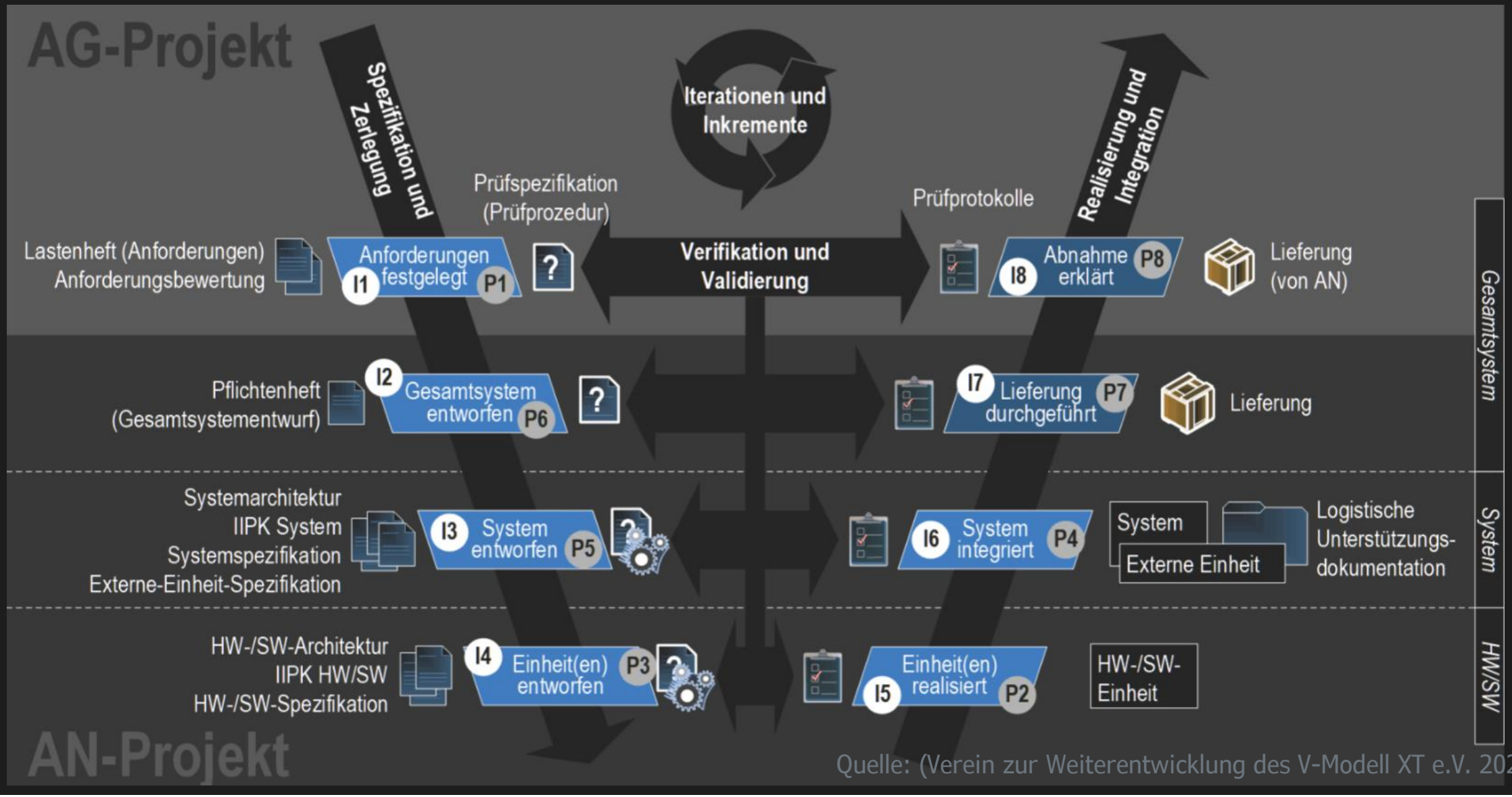
Abschnitt 2.2

V-Modell

Hintergrund

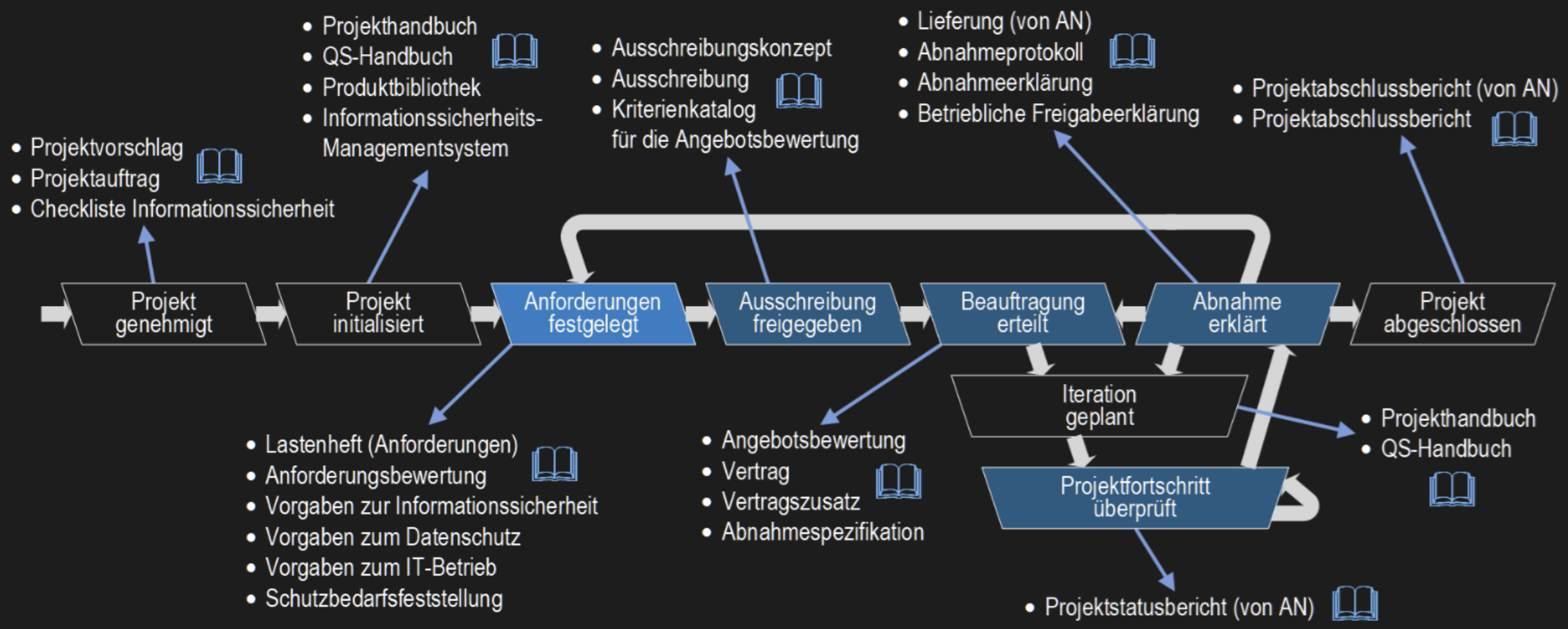
- V-Modell: »vollständiges« Vorgehensmodell
  - Geschützte Marke der Bundesrepublik Deutschland
  - Referenzmodell für Systementwicklungsprojekte, oft vorgeschrieben bei öffentlichen Ausschreibungen
- Ziele
  - Verbesserung der Kommunikation zwischen allen Beteiligten
  - Minimierung der Projektrisiken
  - Gewährleistung von Qualität
  - Eindämmung der Gesamtkosten
  - Verbesserung der Informationssicherheit
- Historie
  - 1992: Erste Version durch Bundeswehr
  - 1997: V-Modell® 97 (inklusive Objektorientierung)
  - 2005: V-Modell® XT (für »Extreme Tailoring«)
  - Aktuell: V-Modell® XT, Version 2.4 (2024)
- Frei nutzbar (Apache License 2.0)

Überblick



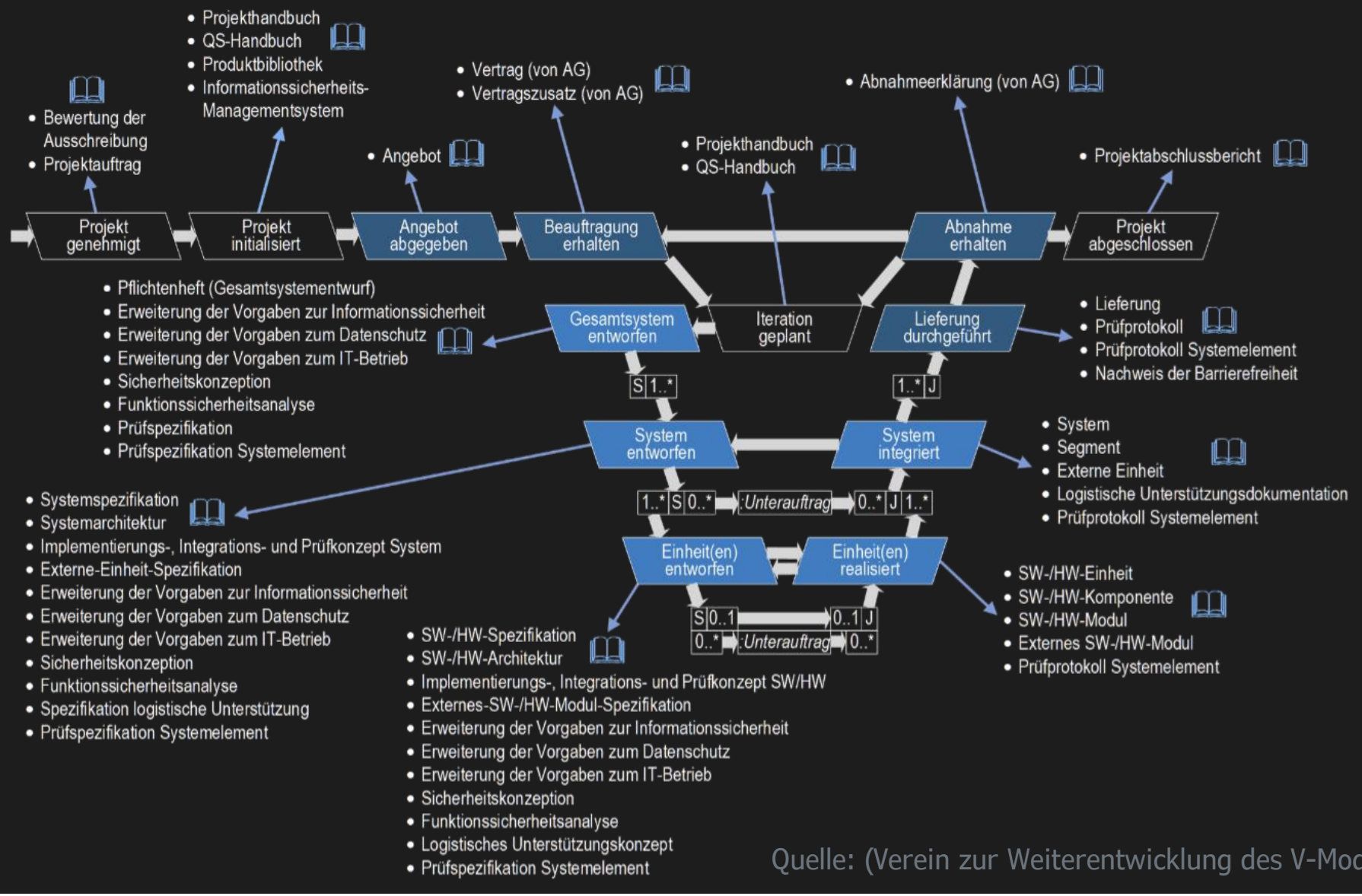
Quelle: (Verein zur Weiterentwicklung des V-Modell XT e.V. 2024)

Auftraggeber: Produkte



Quelle: (Verein zur Weiterentwicklung des V-Modell XT e.V. 2024)

Auftragnehmer: Produkte



Quelle: (Verein zur Weiterentwicklung des V-Modell XT e.V. 2024)

Bewertung des V-Modell® XT

- Plan- und dokumentengetrieben
- Vorteile
  - Klare Strukturierung, transparente Prozesse & Verantwortlichkeiten
  - Hohe Gewichtung von Projektmanagement & Qualitätssicherung
  - Öffentlich zugänglich, ohne Lizenzkosten
  - Generische Baukastenstruktur
  - Anpassbar für Unternehmen bzw. konkretes Projekt (Tailoring)
- Nachteile
  - Sehr umfangreich & komplex, oft zu schwerfällig in der Praxis
  - 6 Projektdurchführungsstrategien, 21 Entscheidungspunkte, 32 Rollen, >100 Produkte etc.
  - Erfordert viel Disziplin von allen Beteiligten
  - Eher ungeeignet für kleinere und mittlere Projekte

Abschnitt 2.3

Agile Vorgehensmodelle

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Softwareentwicklung in einer VUCA-Welt

V

- Volatility (Unstetigkeit)
- Viele & häufige Veränderungen
- Große Schwankungsbreite

U

- Uncertainty (Unsicherheit)
- Unbekanntheit von Abhängigkeiten & Einflüssen
- Unvorhersagbarkeit von Ereignissen & Wirkungen

C

- Complexity (Komplexität)
- Unüberschaubare Zahl von Elementen & Aspekten
- Hochgradige Abhängigkeiten
- Ursache & Wirkung selten eindeutig

A

- Ambiguity (Mehrdeutigkeit)
- Je nach Beobachter unterschiedliche Beschreibungen & Bewertungen
- Keine einfachen Erklärungen

Manifest für Agile Softwareentwicklung, 2001: Werte

Individuen und Interaktion	ist wichtiger als	Prozesse und Werkzeuge
Funktionierende Software	ist wichtiger als	Umfassende Dokumentation
Zusammenarbeit mit Kunden	ist wichtiger als	Vertrags-verhandlungen
Reagieren auf Veränderung	ist wichtiger als	Das Befolgen eines Plans

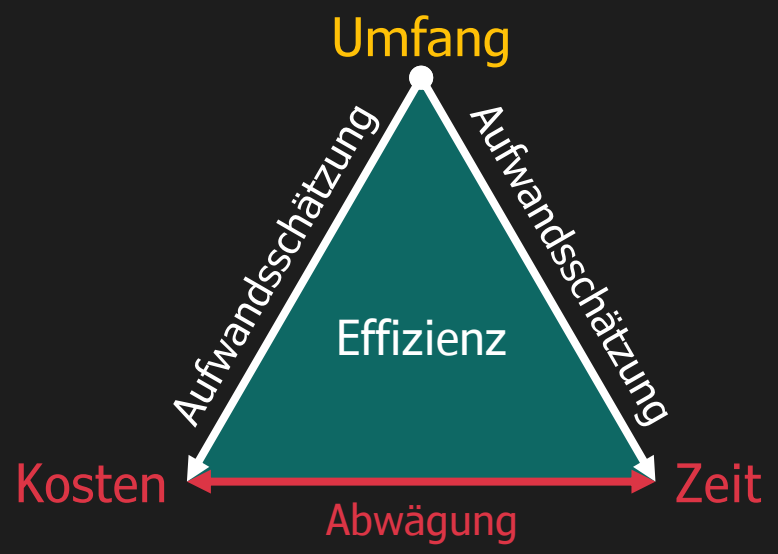
Quelle: (Beck et al. 2001)

Agile Softwareentwicklung

- Bekannte agile Vorgehensmodelle, z.B.
  - Agile Unified Process (AUP), Behavior-Driven Development (BDD), Extreme Programming (XP), Feature-Driven Development (FDD), Lean Software Development, Kanban, Rapid Application Development (RAD), Scrum, Test-Driven Development (TDD) u.v.m.
- Agile Modelle sind oft keine vollständigen »Prozessmodelle«
  - Fokus auf die eigentliche Softwareentwicklung
  - Fehlende Aussagen zu Projektmanagement, Requirements Engineering, Testen etc.
- Agilität heißt nicht »formlos und ohne Regeln«
  - Ebenfalls Ergebnisse: aber geringere Anzahl, weniger Formalismus
  - Ebenfalls Prozesse: aber mehr Spielraum für Alternativen und Kreativität
  - Ebenfalls Verantwortlichkeiten: aber weniger notwendige Rollen
  - Ebenfalls Feedback & Iterationen: aber Rückmeldungen müssen gegeben und eingefordert werden

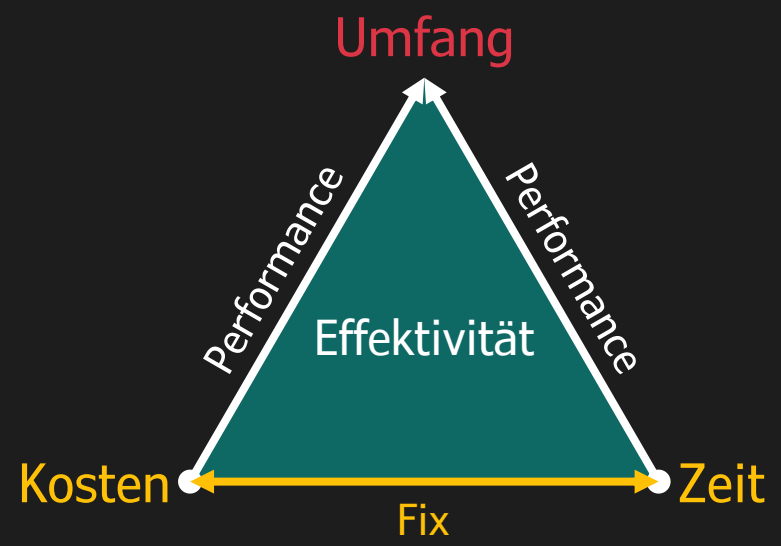
Klassisch versus agil: Projektmanagement

Klassisch (prädikativ)



Umfang ist vorgegeben; Umsetzung mit geringstem Ressourceneinsatz (Zeit & Team)

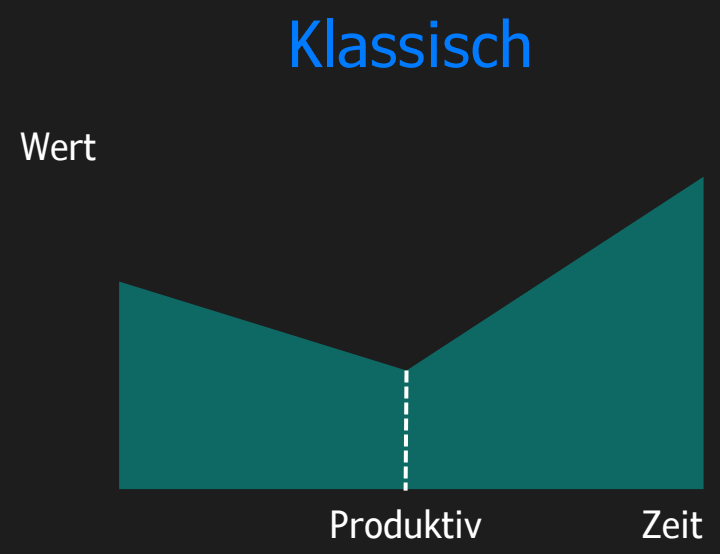
Agil (adaptiv)

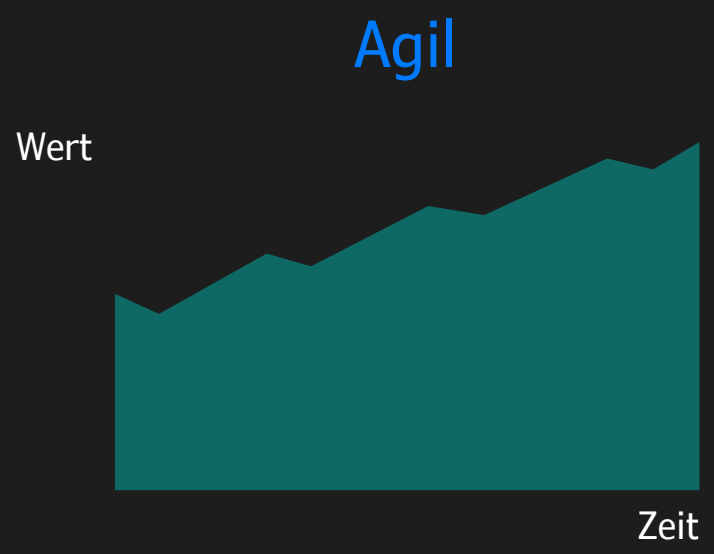


Ressourcen (Zeit & Team) sind pro Iteration vorgegeben (aber nicht insgesamt!); iterative Umsetzung mit jeweils größtem Nutzen

Quelle: (Böhm 2019)

Klassisch versus agil: Wertentwicklung



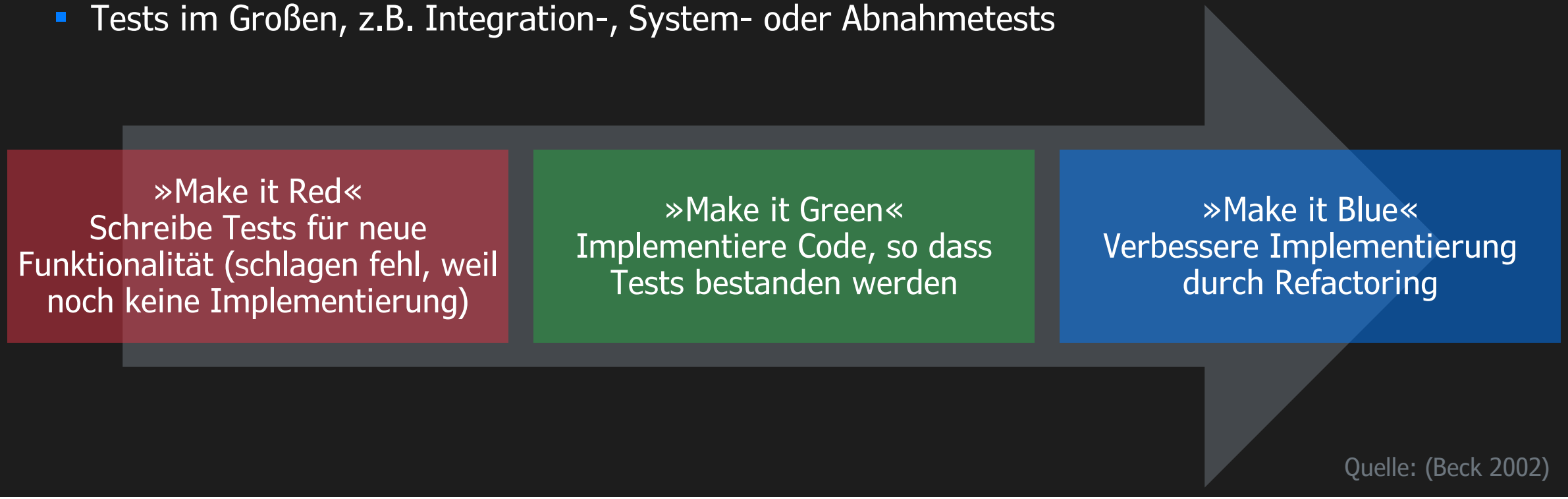


- Gesamtwert wird in beiden Fällen erst beim Projektende realisiert
- Bei agilem Ansatz:
  - Wert wird kontinuierlich geliefert
  - Verzögerungskosten (Cost of Delay) werden reduziert
  - Ggf. auch höherer Gesamtwert, da auf neue Erkenntnisse schneller reagiert werden kann
- Ziel: Time-to-Market bzw. Time-to-Value verkürzen

Quelle: (Westphal 2021)

Test-Driven Development (TDD)

- Ursprung: Kent Beck, 2002 und Extreme Programming
- Erst werden die Testfälle präzisiert und möglichst automatisiert, dann folgt die Entwicklung auf Basis der Testfälle
- Entwicklung mit Fokus, die Tests zu erfüllen
  - Tests im Kleinen, z.B. Komponententests (Unit Tests)
  - Tests im Großen, z.B. Integration-, System- oder Abnahmetests



Quelle: (Beck 2002)

Test-Driven Development (TDD): Vor- & Nachteile

Vorteile

- Sehr genaue Spezifikation
- Erlaubt Test-Automatisierung (wichtig für Continuous Integration)
- Ermöglicht eindeutige »Definition of Done«
- Exakte Messung des Fortschritts bzw. Grad der Umsetzung

Nachteile

- Nur funktionale Anforderungen
- Keine vollstände Spezifikation
- Nicht alles ist ausreichend testbar bzw. automatisierbar
- Erstellung geeigneter Testfälle ist aufwändig
- Für Stakeholder sehr komplex
- Gefahr, dass »Make it Blue« ignoriert wird

Behavior-Driven Development (BDD)

Cucumber
Supported by SMARTBEAR

- Dan North, 2003: Weiterentwicklung von TDD; Kombination mit Domain-Driven Design (DDD)
- Fokus auf »Szenarien«: in der Sprache der Stakeholder formuliert, als Basis für die Erstellung von formalen Testfällen
- Outside-In Development: Einbeziehung der Stakeholder
- Feste Satzschablonen, z.B. Beschreibungssprache Gherkin mit Schlüsselwörtern Given, When, Then, And, Not etc.
- Beispiel: Szenario für Nachklausur
  - Given Zu prüfende Person hat die Klausur nicht bestanden
  - And Es war der Erstversuch
  - When Zu prüfende Person besteht die Wiederholung der Klausur
  - Then Zu prüfende Person hat die Prüfungsleistung erbracht

Quelle: (North 2006)

Refactoring

Refactor – To restructure software code without altering its behavior for the purpose of improving quality attributes, easing future extension or adaptation, or adhering to an architectural style.

(ISO/IEC/IEEE 24765:2017)

- Ralph Johnson, William Opdyke (1990)
- Code ist nie perfekt: kontinuierliche Verbesserung wichtig
- Kompromisslos & als tägliches Geschäft
- Martin Fowler: Catalog of Refactorings, z.B.
  - Change Function Declaration
  - Combine Functions into Class
  - Encapsulate Variable
  - Extract Function
  - Introduce Parameter Object
  - Rename Variable

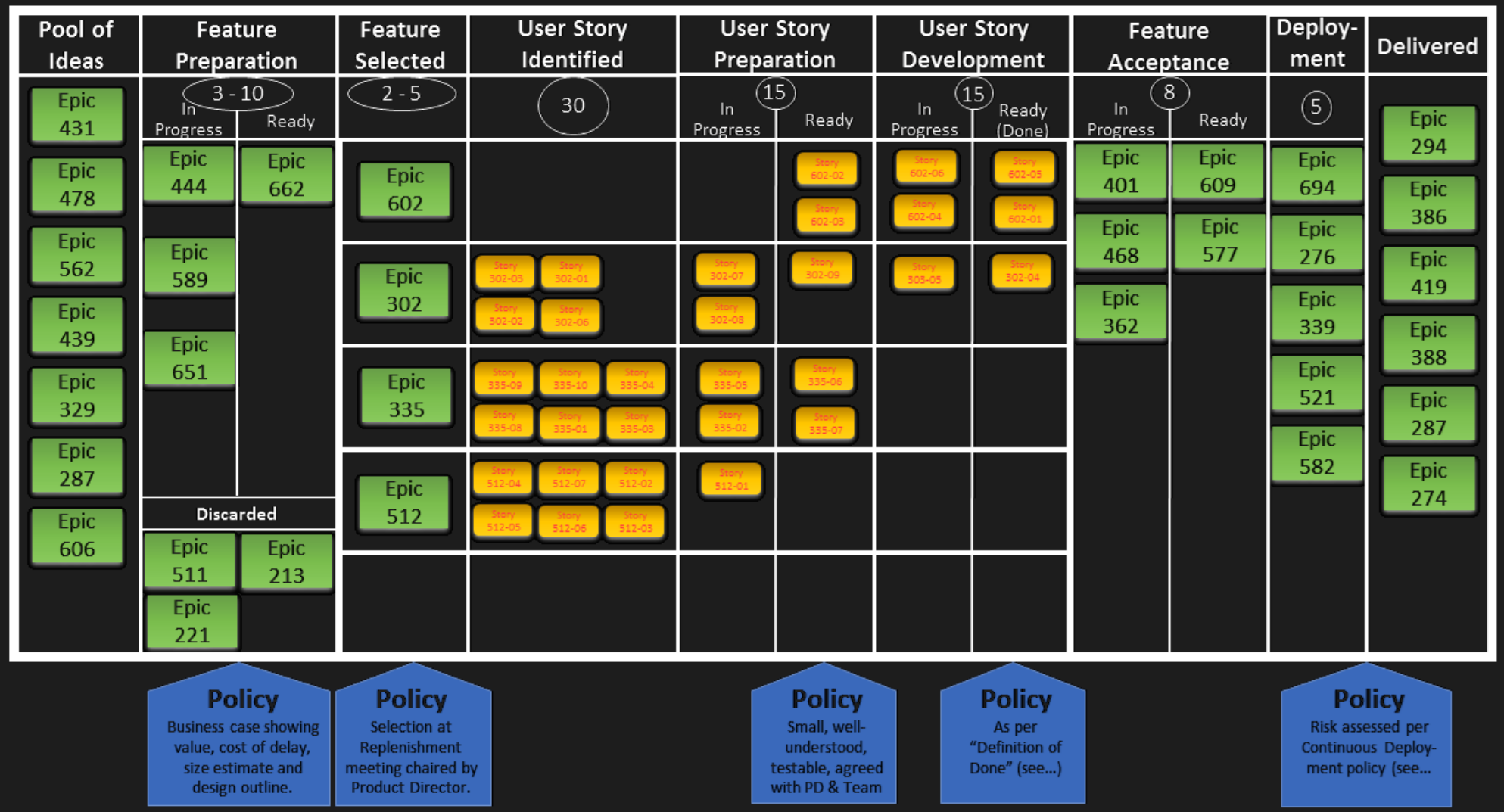
Quelle: (Johnson, Opdyke 1993; Fowler 2020)

Kanban

- Ursprung: Toyota-Produktionssystem, Lean Production, Taiichi Ōno (ab 1950)
- Kanban (japanisch: Signalkarte)
  - Karten wandern auf Kanban Board durch Stationen, Flow von »links nach rechts«
  - Pull-Prinzip: jede Station holt sich Tickets vom Vorgänger ab
  - Anzahl der Tickets pro Station sind begrenzt (Work in Progress, WiP)
- Methode: Fluss messen, steuern & visualisieren (Durchlaufzeit, Durchsatz etc.)
- Ziel: gleichmäßiger Fluss
- Übertragung auf Software Engineering: David J. Anderson, 2007
- Achtung: Unterschied zum Wasserfall-Modell
  - Wasserfall: gesamtes Produkt durchläuft eine Wasserfall-Stufe
  - Kanban: Teile des Produkts durchlaufen die Stationen
- Kanban wird oft bei agile Softwareentwicklung als Instrument eingesetzt, insbesondere auch bei Scrum

Quelle: (Anderson 2010; Vacanti, Yeret 2021)

Kanban Board

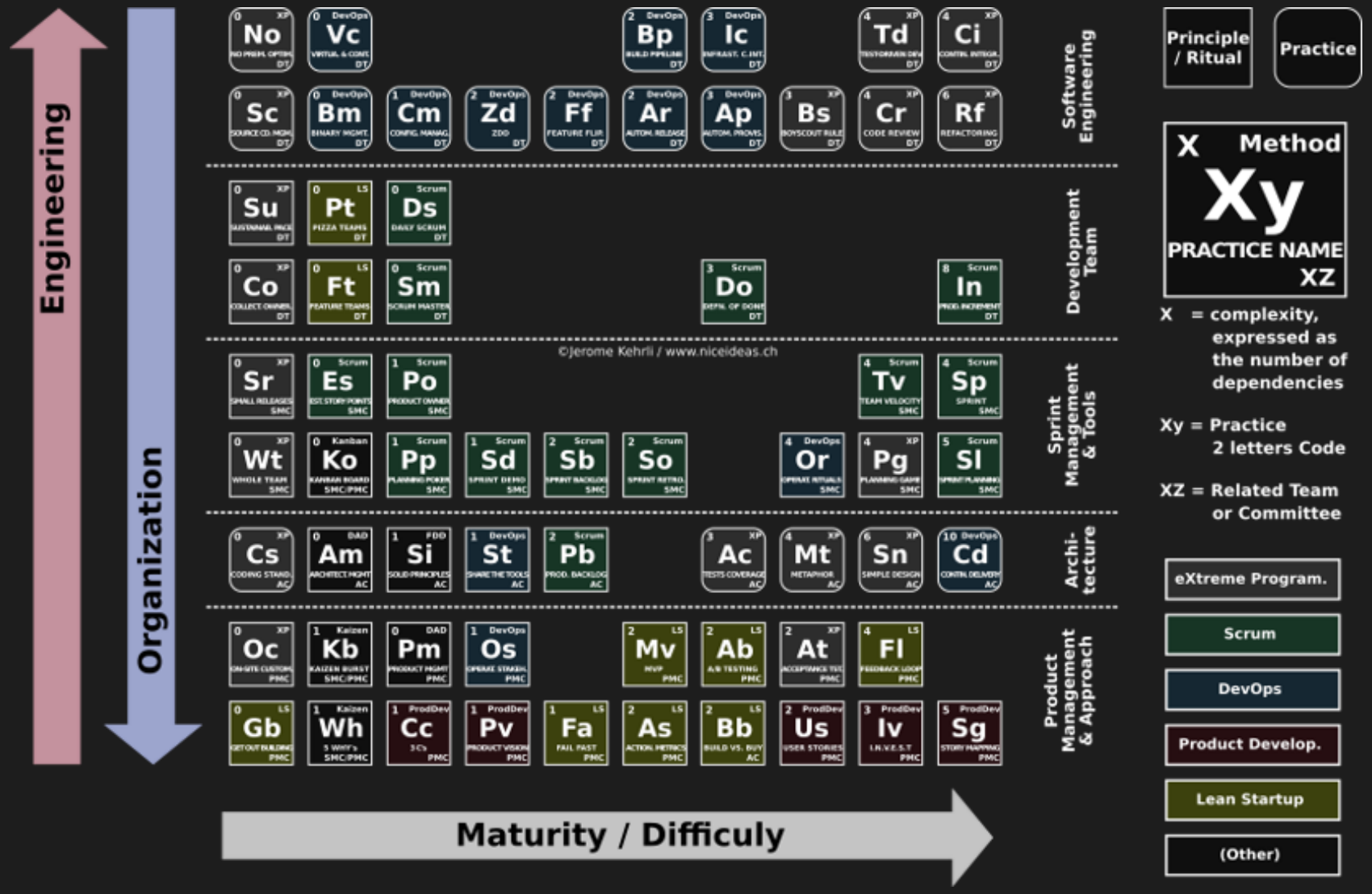


Weitere agile Prinzipien

- Viele Einflüsse insbesondere durch Extreme Programming (XP) von Kent Beck, Ward Cunningham, ab 1995
- Pair Programming
  - Vier-Augen-Prinzip: zwei Entwickelnde, eine IDE; abwechselnd
  - Mehr als zwei Entwickelnde: Mob Programming
- Collective Code Ownership
  - Gemeinsame Verantwortung im Team; keine Wissensmonopole
- System Metaphor
  - Gemeinsame »Story«, erzählt von allen Stakeholdern
- Programmer Welfare / Sustainable Pace

Quelle: (Beck, Andres 2004)

Periodic Table of Agile Principles and Practices



Quelle: (Kehrli 2017)

Wann ist Agilität sinnvoll?

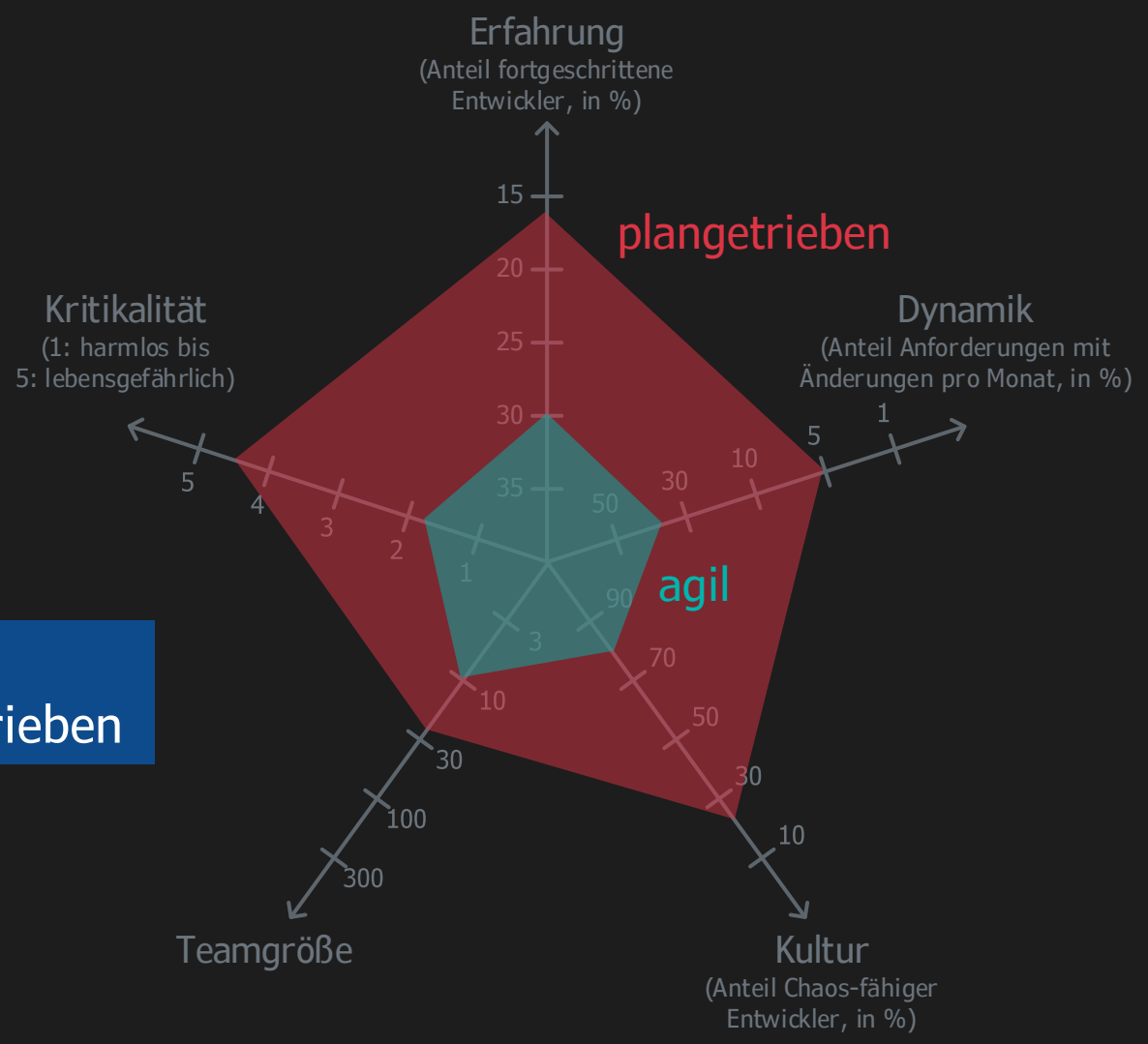
Sinnvoll

- Wunsch nach Agilität bei Auftraggebenden
- Bedarf nach schneller Auslieferung eines ersten Inkrements
- Entwicklung neuartiger Lösungen
- Bewährte Zusammenarbeit von Auftraggebenden & Auftragnehmenden
- Interne Projekte & neue Releases

Nicht sinnvoll

- Andere Vorgaben, z.B. öffentliche Projekte
- Vollständige Planbarkeit nötig, z.B. Festpreisprojekt
- Auftraggebende / Auftragnehmende nicht geeignet oder unwillig
- Software nicht geeignet
- Geeignete Experten nicht verfügbar

Entscheidungsmodell: agil versus plangetrieben



Kleine Fläche → agil
Große Fläche → plangetrieben

Quelle: nach (Boehm, Turner 2003)

Rechtliche Aspekte

- Auch anfangs gute agile Projekte können zum Rechtsstreit zwischen Auftraggebenden & Auftragnehmenden führen
- Unklare Spezifikation führt oft zu rechtlicher Problematik
- Gewährleistungspflicht (3 Jahre ab Abnahme nach BGB): bei iterativem Vorgehen nicht eine, sondern viele Abnahmen
- Haftungs-Problematik
  - Haftungsanspruch besteht bei fahrlässigen Fehlern in der Software
  - Aber: wann kann man bei Agilität von Fahrlässigkeit sprechen?
  - Wie kann man diese bei geringer Dokumentation feststellen?
  - Wer haftet? Das Entwicklungsteam? Der Product Owner?
- Immer vor Projektbeginn schriftlich vereinbaren
  - Welche Arten von Anforderungen sind vertraglich bindend (»Must-Have«) und welche sind nur Diskussionsgrundlage?

Abschnitt 2.4

Scrum

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Hintergrund

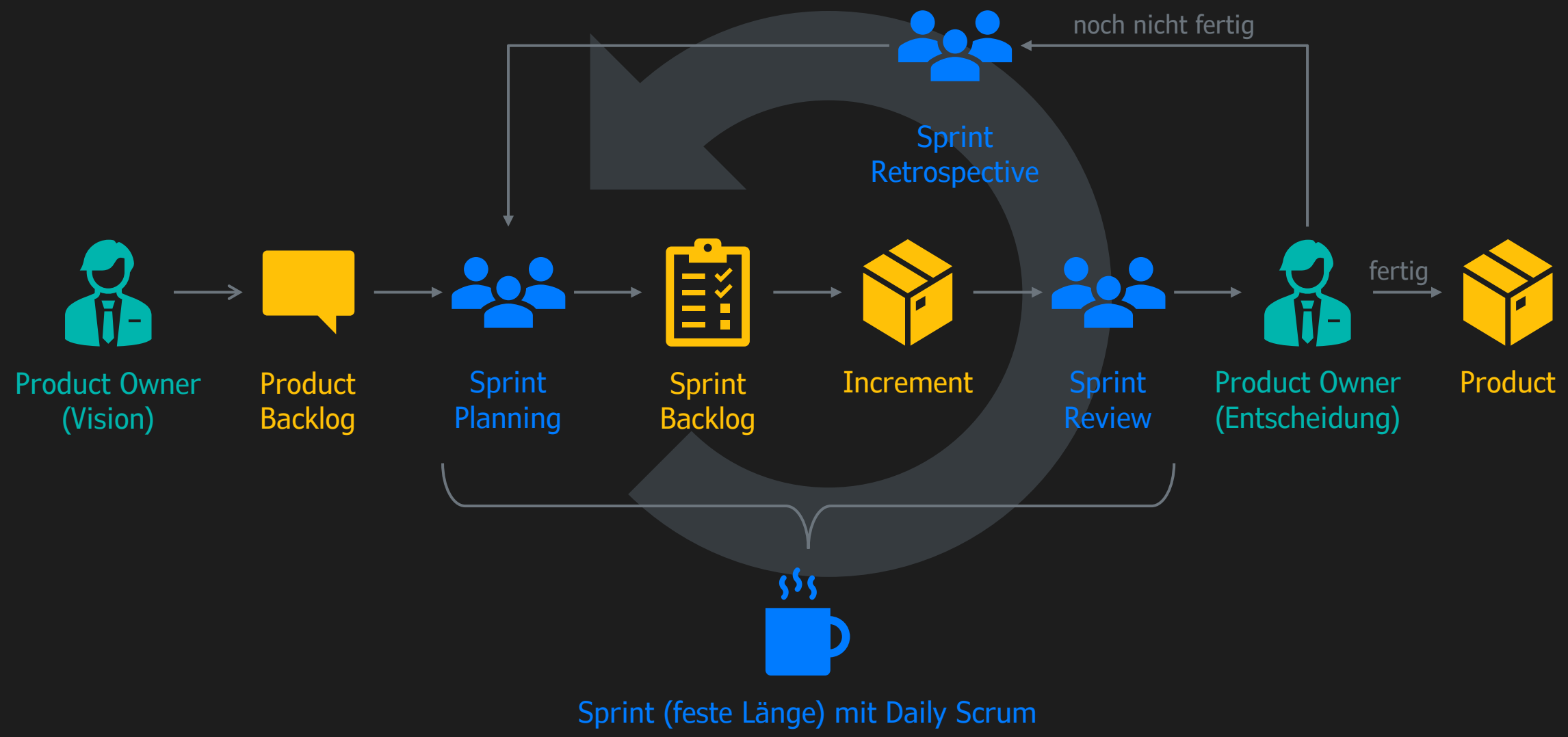
- Ken Schwaber, Jeff Sutherland, Mike Beedle (~1995)
  - »Leichtgewichtiges Rahmenwerk, welches Menschen, Teams und Organisationen hilft, Wert durch adaptive Lösungen für komplexe Probleme zu generieren«
- Basiert auf Lean Thinking (Takeuchi, Nonaka 1986)
- Scrum ~ Rugby-Spieleröffnung (»Gedränge«)
- Iterativ & inkrementell, rollenbasiert

- Ansatz
  - Empirische Prozesssteuerung: akzeptieren, dass Problem vorab nicht vollständig verstanden wird & Auftraggebende ihre Meinung ändern
  - Ganzheitlich: Team als Einheit, um gemeinsames Ziel zu erreichen
  - Prognosesicherheit optimieren & Risiken kontrollieren
- Drei Säulen: Transparenz, Überprüfung & Anpassung
  - Fortschritt regelmäßig überprüfen, um unerwünschte Abweichungen aufzudecken
  - Prozess / Produkt so schnell wie möglich anpassen, wenn Abweichungen festgestellt werden

Überblick

- Scrum Team: 1x Product Owner, 1x Scrum Master, mehrere Developer
  - Selbstorganisiert & interdisziplinär: Team entscheidet selbst, wie die Arbeit erledigt wird
  - Alle nötigen Kompetenzen; keine Hierarchie; keine Abhängigkeit von Personen außerhalb
  - Umsetzungs- & ergebnisverantwortlich (Responsible & Accountable)
- Zugehörende Artefakte, Ereignisse & Regeln
- Kurze, zeitlich begrenzte Sprints (Iteration, 2 bis 4 Wochen)
- Inkrement als potenziell auslieferbares Produkt am Ende jedes Sprints
- Überprüfung durch den Markt [→ Validated Learning]
- Time Boxing
  - Sprint-Dauer ist fixiert und darf nicht verlängert werden
  - Alle Ereignisse haben feste zeitliche Beschränkung (sie dürfen aber vorzeitig beendet werden, sobald sie ihren Zweck erfüllt haben)

Ablauf


noch nicht fertig
Sprint Retrospective
Product Owner (Vision)
Product Backlog
Sprint Planning
Sprint Backlog
Increment
Sprint Review
Product Owner (Entscheidung)
fertig
Product
Sprint (feste Länge) mit Daily Scrum

Scrum Team

Product Owner (PO)

- Einzelne Person, kein Komitee
- Repräsentiert alle Stakeholder
- Verantwortlich für Maximierung des Produktwerts
- Definiert & priorisiert Anforderung
- Verantwortlich für Verwaltung & Verständnis des Product Backlog
- Organisation muss Entscheidungen respektieren

Scrum Master (SM)

- Verantwortlich für Effektivität des Scrum Teams
- Sorgt dafür, dass Scrum richtig angewendet wird
- Coach
- Beseitigt Hindernisse (Impediments), die das Team aufhalten
- Vermittelt zwischen Scrum Team & Stakeholdern

Developer

- Experten:innen, die Inkremente selbst herstellen
- Selbstorganisiert & interdisziplinär
- Keine Titel, keine Hierarchie, alle sind Developer
- Optimale Größe: max. 8 Personen
- Verantwortlich für Planung & Qualität

Artefakte

Product Backlog (PB)

- Commitment: Produkt-Ziel
- Geordnete Liste von Anforderungen
- Dynamisch (Refinement)
- Alle beteiligt, aber PO verantwortlich
- Details: nur für Planung des nächsten Sprints
- Je weiter oben, desto detaillierter
- Good Practice: Value Points

Sprint Backlog (SB)

- Commitment: Sprint-Ziel
- Auswahl aus PB von oben
- Pull: Developer bestimmen Anzahl (Forecast & Velocity)
- Verwaltet durch Developer, sie definieren Tasks
- Transparent, dynamisch, aktuell

Increment

- Commitment: Definition of Done
- Konkreter Schritt Richtung Produkt-Ziel
- Additiv & geprüft
- Verwendbar & potenziell auslieferbar
- Mehrere Inkremente pro Sprint & vorzeitige Lieferung sind möglich

Ereignisse (1)

Sprint

- Feste Länge
- Keine Änderungen, die Sprint-Ziel gefährden
- Qualität nimmt nicht ab
- PB wird bei Bedarf verfeinert
- Inhalt & Umfang kann mit PO neu vereinbart werden
- Nach Abschluss beginnt unmittelbar neuer Sprint
- Nur PO darf Sprint abbrechen

Sprint Planning

- Gesamtes Scrum Team
- Am Anfang jedes Sprints
- Thema 1: Wofür? Sprint-Ziel klären
- Thema 2: Was? Developer bestimmen im Gespräch mit PO den SB
- Thema 3: Wie? Developer planen Umsetzung durch Tasks
- Max. 120 Min / Sprintwoche

Daily Scrum

- Gesamtes Scrum Team, ggf. ohne SM
- Überprüfung & Kommunikation
- An jedem Arbeitstag: gleiche Zeit & Ort
- 15 Minuten
- Jeder Developer berichtet:
  - Was habe ich seit gestern gemacht?
  - Was plane ich für heute?
  - Welche Hindernisse?

Ereignisse (2)

Sprint Review

- Gesamtes Scrum Team, eventuell weitere Stakeholder
- Echte Demonstration des Increment
- Was wurde im Sprint erreicht? Was nicht?
- Welche Anpassungen sind in Zukunft nötig?
- Einzige Situation, bei denen Stakeholder formal integriert sind
- Max. 60 Min / Sprintwoche

Sprint Retrospective

- Developer & SM, aber ohne PO
- Reflektion: kontinuierliche Prozess-verbesserung
- Was lief gut im letzten Sprint? Was kann im nächsten Sprint verbessert werden?
- Betrifft: Personen, Interaktionen, Prozesse, Werkzeuge, Definition of Done
- Nächster Sprint Backlog sollte immer eine Verbesserung enthalten (Team Story)
- Max. 45 Min / Sprintwoche

Hilfsmittel: Kanban Board

Cloudy_Venus Team

New Work Item | Column Options

Taskboard | Backlog | Capacity | Analytics

Q3.2024 | Person: All

1. Juli - 30. September
9 work days remaining

Expand all | New 156 | Active 32 | Closed

As a lecturer, I can show examples of all GoF design patterns — 8 not started

263 As a developer, I can use interfaces for all number representations, so that there is a uniform and generic access to these numbers
12
Story Poi... 20
Developer Story

281 Use generic fraction instead of In32Fraction
6
Original E... 8
Complete... 2

340 Add custom JSON converter for Fraction
Philipp Rohde 2
Original E... 2

341 Add custom JSON converter factory for generic fractions
Philipp Rohde 4
Original E... 4

Kanban bei Scrum nicht vorgeschrieben, aber sinnvoll

As a developer, I can use units and unit values so that I can use them in linear optimization applications — 3 in progress

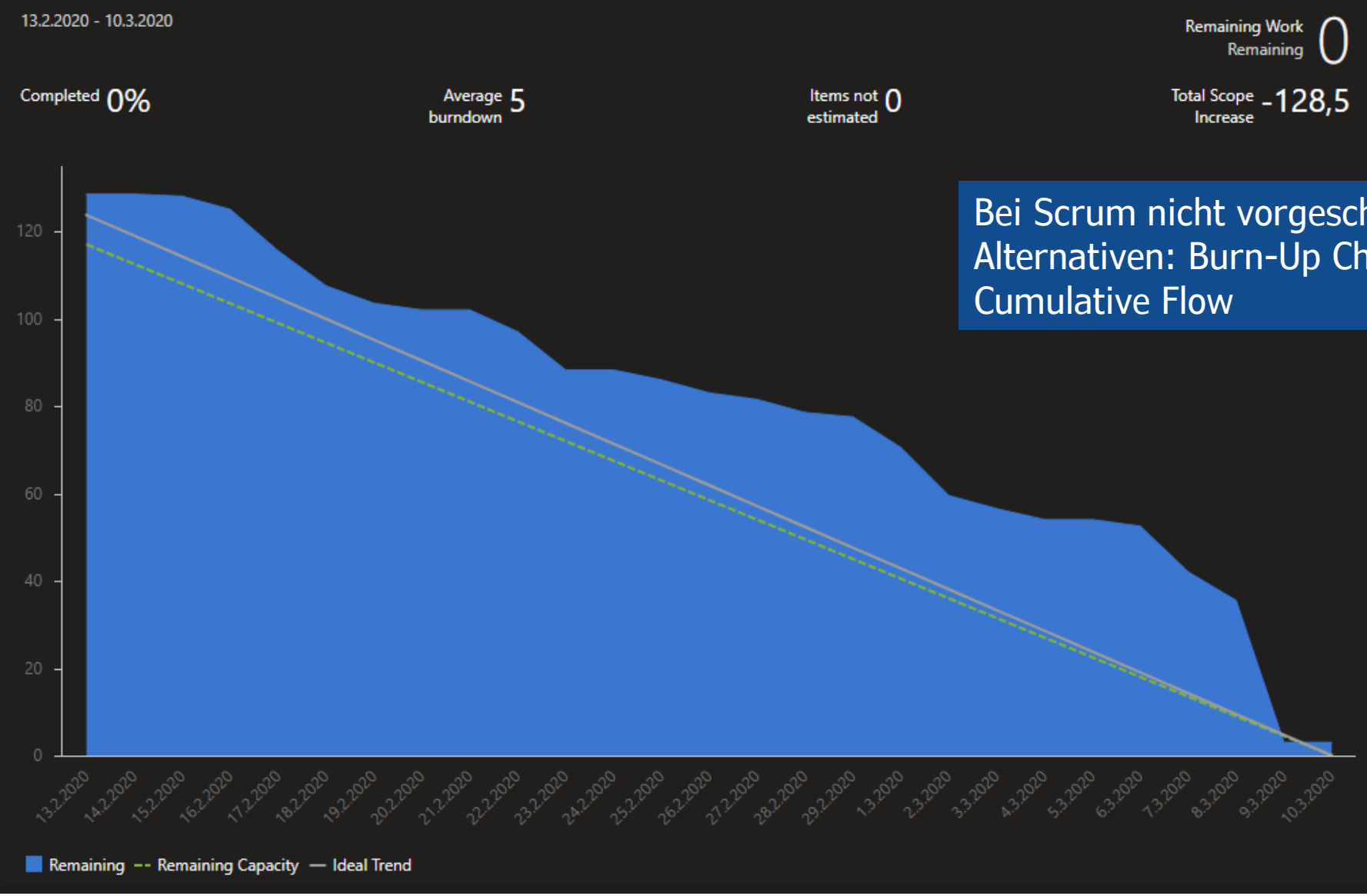
As a lecturer, I can show examples of linear problems to demonstrate an application of linear optimization — 8 not started

As a lecturer, I can show examples of transport problems to demonstrate an application of linear optimization — 5 not started

As a lecturer, I can show examples of assignment problems to demonstrate an application of linear optimization — 4 not started

As a developer, I can use graphs to implement them in graph algorithms — 2 in progress

Hilfsmittel: Burn-Down Chart



Bei Scrum nicht vorgeschrieben
Alternativen: Burn-Up Chart, Cumulative Flow

Definition of Ready (DoR)

- Vor dem Sprint
- Präzisiert, wann ein Arbeitsprodukt aus PB im Sprint bearbeitet werden darf
- Verbindlich für alle im agilen Team; oft organisationsweit einheitlich
- Beispiele für eine User Story
  - Ist die Beschreibung eindeutig und präzise?
  - Sind alle Qualitätskriterien (z.B. INVEST) erfüllt?
  - Sind alle Pflichtangaben vorhanden, insbesondere Akzeptanzkriterien & Story Points?
  - Sind die Auswirkungen auf alle anderen Komponenten geklärt?
  - Sind alle notwendigen UML-Diagramme vorhanden?
  - Sind genügend Testfälle spezifiziert?

Definition of Done (DoD)

- Nach dem Sprint
- Präzisiert, wann ein Arbeitsprodukt abgeschlossen ist
- Definiert allgemeine Rahmenbedingungen & Qualitätsanforderungen
- Verbindlich für alle im agilen Team; oft organisationsweit einheitlich
- Zwei Sichten
  - Arbeitsprodukt: welche Eigenschaften muss es besitzen?
  - Prozess: welche Tätigkeiten müssen erledigt sein?
- DoD auf verschiedene Ebenen möglich
  - Allgemein für Arbeitsprodukte (z.B. User Stories) oder für Sprint, Release, Codierung, Design / Architektur, Tests, Rollout etc.
- DoD macht typischerweise Aussagen zu Coding Conventions, Dokumentation, Tests & Testabdeckung, Code Reviews
- DoD muss präzise anwendbar sein

DoD: Beispiel

- Alle Einträge im Sprint Backlog haben den Status »Done«
- Quellcode ist in Trunk eingecheckt & Build war erfolgreich
- Unit Tests
  - Alle Tests der Kritikalität 1 sind automatisiert
  - Testabdeckung insgesamt größer als 95 %
  - Keine Fehler der Klassen 1 oder 2
  - Maximal 5 Fehler der Klasse 3
- Integrations- und Systemtest gemäß Spezifikation erfolgreich
- Codereview für alle neuen Teile gemäß Spezifikation, z.B.
  - Coding Conventions
  - Code Review durch einen weiteren Entwickelnden (mind. Level 2)
- Dokumentation gemäß Spezifikation, z.B.
  - Konfigurations- und Parameterdokumentation
  - Nutzungs-, Installations- und Administrationshandbuch
  - Inline-Codedokumentation

Scrum: Anti-Pattern

- Sprintlänge
  - Kurze Sprints (weniger als 3 Wochen): zu wenig Zeit, um wirklich etwas zu implementieren; zu viel Overhead: schlechtes Verhältnis von Berichten bei Ereignissen (Sprint Planning, Sprint Review etc.) zu tatsächlichem Implementieren
  - Lange Sprints (mehr als 6 Wochen): zu wenig Feedback & Prüfung durch Kunden; zu wenig agile Dynamik
  - Beachte: eine dynamische Anpassung der Sprintlänge ist nicht erlaubt
- Teamgröße
  - Kleine Teams (weniger als 5 Personen): zu viel Overhead; direkte Abstimmung effektiver
  - Große Teams (mehr als 15 Personen): zu viel Reibungsverluste durch Kommunikation, Abstimmung, Kontrolle; eher wieder klassisches Projektmanagement nötig
  - Beachte: das Team sollte fest bleiben, Experten können bei Bedarf hinzugezogen werden
  - Keinesfalls sollte man zu verspätetem Projekt unbedarft weitere Personen hinzufügen, um die Ziele zu erreichen (Brooks'sches Gesetz: »Adding manpower to a late project makes it later«)

Problematische Teammitglieder

- Scrum funktioniert nur, wenn
  - Product Owner ist fachlich kompetent und entscheidungsfähig
  - Entwicklungsteam fragt so lange, bis alle Unklarheiten beseitigt sind
  - Scrum Master berät und unterstützt kompetent
- Schwierige Typen als Product Owner
  - »Visionär«: Sieht nur die großen Ziele; vernachlässigt Detailarbeit
  - »Teilzeit-Product-Owner«: zu viele parallele Aufgaben; zu wenig Zeit
  - »Product-Owner-Proxy«: kann oder darf nicht eigenständig entscheiden
- Schwierige Typen als Scrum Master
  - »Geheimer Projektleiter«: glaubt nicht an Agilität, möchte das Team klassisch leiten
  - »Agile Mimose«: kann mit Schwierigkeiten & Problemen nicht umgehen
- Schwierige Typen im Entwicklungsteam
  - »Supergenau«: legt erst los, wenn alles bis ins letzte Detail geklärt ist
  - »Hacker«: das Gegenteil – möchte nicht planen, sondern sofort loslegen

Quelle: (Bergsmann 2017)

Studie: Effektivität von Scrum Teams (2023)

- Analyse von repräsentativen Fallstudien: wesentlich sind 5 Faktoren

Ability of teams to be responsive to the changing needs of stakeholders (frequent releases, effective refinement)

A strong concern for the needs of stakeholders (focus on value, clear sprint goals, collaboration with stakeholders, high-quality sprint reviews)

A climate of continuous improvement (learning environment, strong concern for quality, psychological safety, high-quality sprint retrospectives)

Teams need to have the autonomy to make necessary changes (self-management, cross functionality)

Strong support from management

Quelle: (Verwijs, Russo 2023)

Skalierung von Scrum

- Herausforderung: Skalierung von Scrum
  - Standard-Scrum: ein Entwicklungsteam
  - Erweiterung von Scrum auf mehrere Teams, die an zusammenhängenden Produkten arbeiten
  - Ziel: nicht nur »separate Scrum-Teams plus Zusammenführen«, sondern echte agile Verzahnung der Zusammenarbeit
- Verschiedene Vorgehensmodelle, z.B.
  - Large-Scale Scrum (LeSS, 2005)
    - LeSS (2–8 Teams, je 3–9 Mitglieder)
    - LeSS Huge (mehr als 8 Teams)
  - Nexus Framework (2015)
  - Scaled Agile Framework (SAFe, 2011)

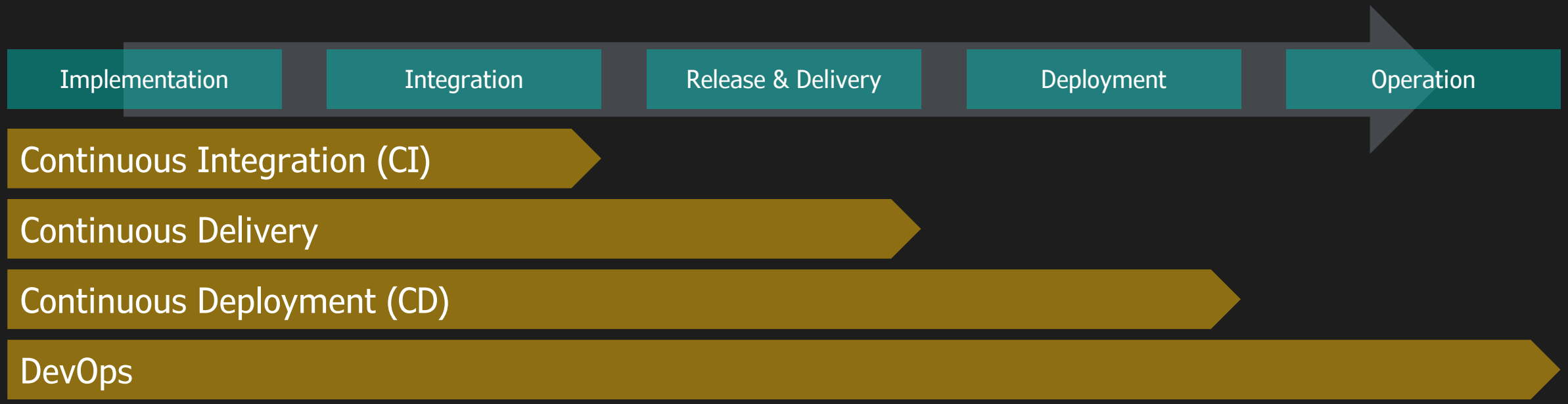
Quelle: (Larman, Vodde 2016; Schwaber 2021; Scaled Agile, Inc. 2025)

Abschnitt 2.5

Continuous Integration & DevOps

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Continuous Integration & Continuous Deployment (CI/CD)



- Fortlaufenden Zusammenführung der Komponenten zu einem System (früher: »Nightly Build«); Ursprung: Kent Beck & Martin Fowler
- Ziele
  - Strukturierung der Anwendung (Feature-Driven Design, FDD); Verkürzung der Releasezyklen
  - Automatisierung der Tests, frühe Entdeckung von Fehlern
  - Ständige Verfügbarkeit von Demo- und Testsystemen
  - Steigerung der Softwarequalität

CI/CD: Prinzipien

- Gemeinsame Codebasis (Versionsverwaltung)
- Automatisierte Übersetzung
  - Einheitlich definierte Tests; statische Überprüfungen für jede Integration
  - Verhindern von »It works on my machine«
- Automatisierte Verteilung & gespiegelte Produktionsumgebung
  - Automatisierte Verteilung in alle Umgebungen (DEV, TEST, PROD)
  - Änderungen werden in Abbild der Produktionsumgebung getestet
- Häufige Integration
  - Reduziert Risiko von Fehlschlägen; sichert gemeinsame Codebasis
  - Kontinuierliche Integration in Mainline (Trunk-Based Development, TBD)
- Kontinuierliche Test-Entwicklung
  - Zu jeder Änderung: Tests, Test-Dokumentation, Code Coverage Analysis
  - Kurze Testzyklen fördern häufige Integrationen
- Dokumentation und einfacher Zugriff
  - Einfacher Zugriff für alle Beteiligten; automatisiertes Reporting

Blue/Green Deployment & Canary Release

- Blue/Green Deployment
  - Rollout neues Release / Feature: betreibe zwei parallele Produktionsumgebungen
  - Blue: alter Stand (»Last-Known Good«); Green: neuer Stand
  - Lenke durch Router Nutzende auf Green; bei Problemen schalte zurück auf Blue (»Rollback«)
  - Verwandt: Feature Toogle, wenn Features im Quellcode aktiviert / deaktiviert werden können
- Canary Release
  - Auch: Phased Rollout, Incremental Rollout
  - Ebenfalls zwei parallele Produktionsumgebungen
  - Lenke durch Router Teil der Nutzenden auf neuen Stand (z.B. Early Adopters), sammle Feedback
  - Restliche Nutzende bleiben auf altem Stand
  - Nach Testphase: alle Nutzenden auf neuen Stand
  - Verwandt mit A/B-Test bzw. Splittest

Quelle: (Humble, Farley 2010)

DevOps: Development & Operations

- Hintergrund: oft tiefe Gräben zwischen Entwicklung und Betrieb
  - Fehlendes Verständnis & Vertrauen (»Wall of Confusion«)
  - Entwicklung: möchte Veränderung, neue Produkte & Features
  - Betrieb: möchte Stabilität, Sicherheit & Verfügbarkeit
- Ziele
  - Softwareentwicklung als »Fließbandfertigung« in einer »IT-Fabrik«
  - Effektivere Zusammenarbeit von Verantwortlichen, Entwickelnden, Testenden und Betreibenden unter Einbeziehung der Nutzenden
  - Überwindung von organisatorischen und mentalen »Silos«
  - Aufeinander abgestimmte Infrastruktur & Prozesse
- Eigenschaften
  - Kein einheitliches Vorgehensmodell, eher das Beste von lean & agil
  - Primär Frage der Kultur, erst dann der Technik

Quelle: (Halstenberg, Pfitzinger, Jestädt 2020)

DevOps: CAMS-Prinzipien

Culture

- Kultur der Zusammenarbeit
- Subsidiarität
- Blame-Free Culture
- Eat Your Own Dog Food
- Chaos Engineering

Automation

- Continuous Integration
- Shift Left

Measurement

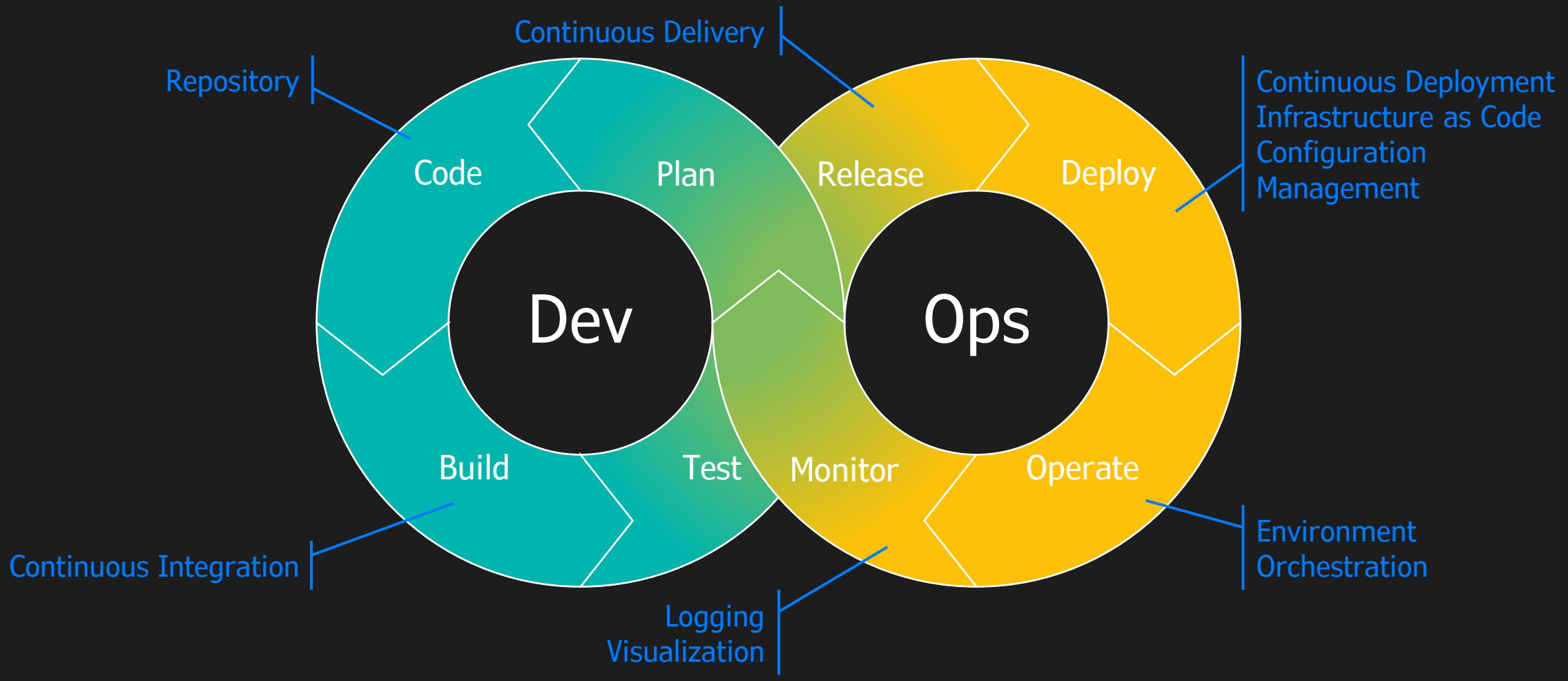
- Service-Level-Agreements (SLA)
- Mean Time To Change (MTTC)
- Mean Time to Recovery (MTTR)
- Mean Time Between Failures (MTBF)

Sharing

- Kommunikation
- Transparenz
- Best Practices

Quelle: (Halstenberg, Pfitzinger, Jestädt 2020)

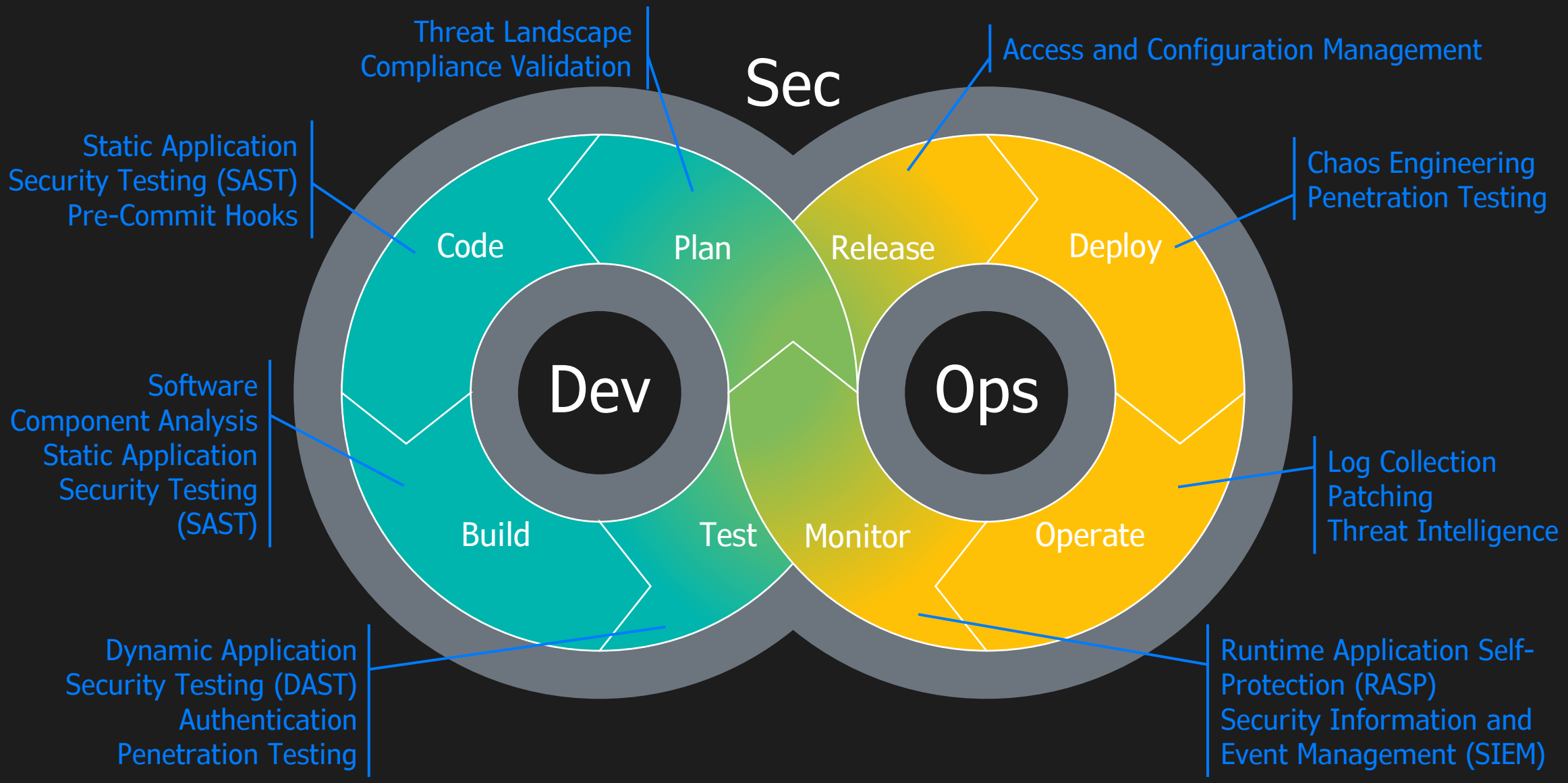
DevOps


Continuous Delivery
Repository
Continuous Deployment
Infrastructure as Code
Configuration
Management
Code
Plan
Release
Deploy
Dev
Ops
Build
Test
Monitor
Operate
Continuous Integration
Environment
Orchestration
Logging
Visualization

DevOps Pipeline

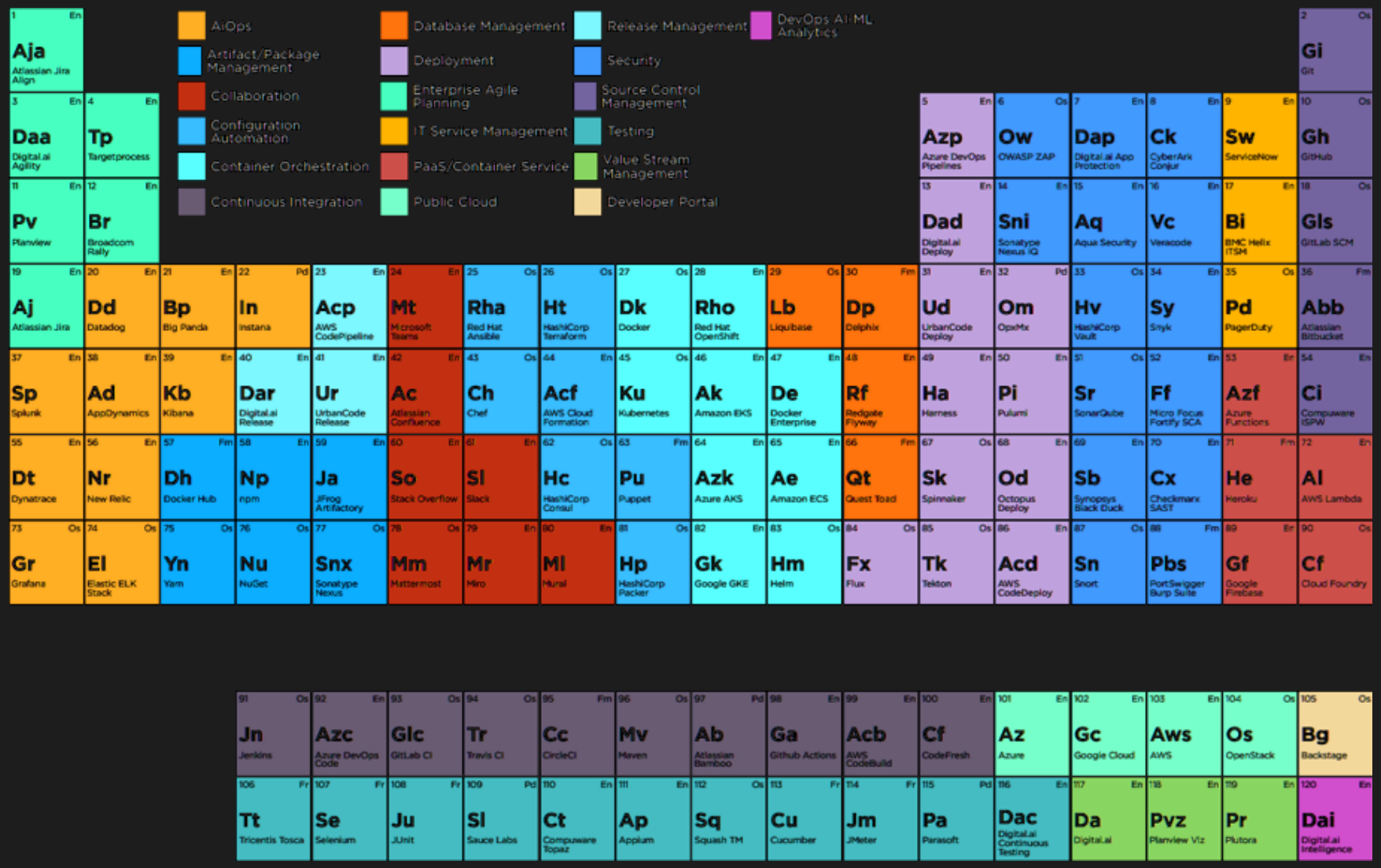
- Validierungen, die Software auf dem Weg zur Veröffentlichung bestehen muss
- Realisierung durch eine Tool Chain; oft Kombination aus vielen Werkzeugen, z.B.
  - Source Control Management: z.B. Git, GitHub, GitLab, Bitbucket [→ Exkurs T]
  - Paketmanagement: z.B. npm, Yarn, NuGet, PyPI
  - Orchestrierung & Steuerung der Pipeline: z.B. Jenkins, GitLab CI, Azure DevOps
  - Build Automation: z.B. Apache Maven, Gradle, Ant [→ Exkurs T]
  - Konfigurationsmanagement / Infrastructure as Code (IaC): z.B. Ansible, Chef, Puppet, Terraform
  - Containervirtualisierung / Containerorchestrierung: z.B. Docker, Kubernetes
  - Testautomatisierung: z.B. JUnit, Selenium, Appium, Cypress, Playwright [→ Exkurs T]
  - Statische Analyse / Security / Continuous Inspection: z.B. OWASP ZAP, SonarQube [→ Kapitel 6]

DevSecOps: DevOps + Security



FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

DevSecOps Tools Periodic Table



Quelle: (Digital.ai 2023)

Abschnitt 2.6

Versionsverwaltung & Rückverfolgbarkeit

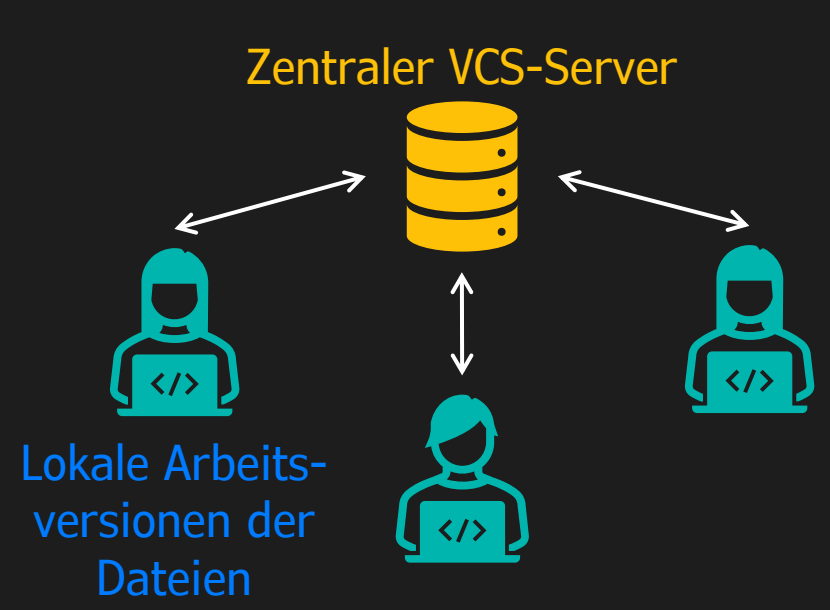
FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Versionsverwaltung

- Versionsverwaltung (Version Control System, VCS) bzw. Quellcodeverwaltung (Source Code Management, SCM)
- Koordinierung des Zugriffs von vielen Entwickelnden
- Protokollierungen von Änderungen (Dateien, Versionen, Autor, Zeitpunkt etc.)
- Vergleich und Wiederherstellung von alten Versionsständen
- Einfrieren spezifischer Version, z.B. Release (Code Freeze)
- Gleichzeitige Entwicklung mehrerer Entwicklungszweige (Branching)
- Zusammenführen einzelner Entwicklungszweige (Merging)
- Verwalten eines Hauptentwicklungszweig (Trunk oder Main)
- Versionsnummern gemäß Semantic Versioning: Major.Minor.Patch, z.B. 8.5.42

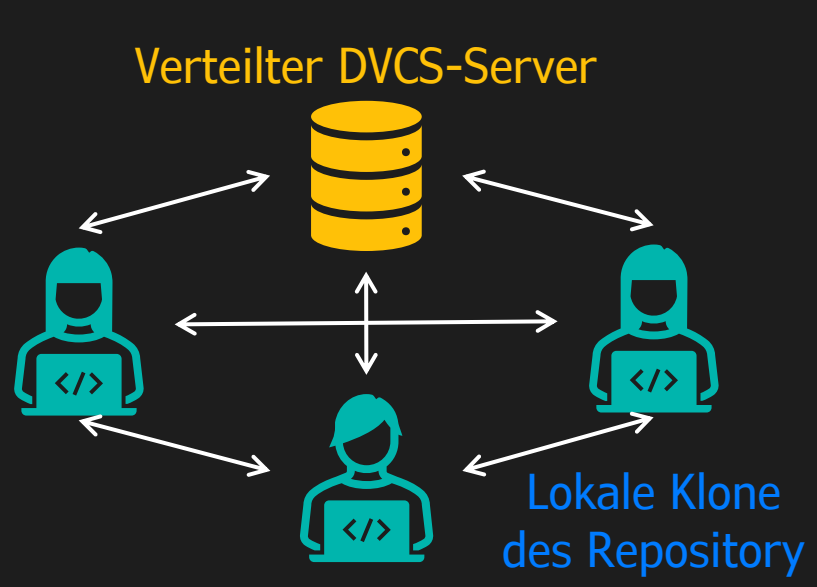
Zentrale & verteilte Versionsverwaltung

Zentral



Zentrales Repository

Verteilt (Distributed)



- Beispiel: Concurrent Versions System (CVS), Subversion (SVN), Microsoft Team Foundation Server (TFS)
- Problem: Performanz, Single Point of Failure

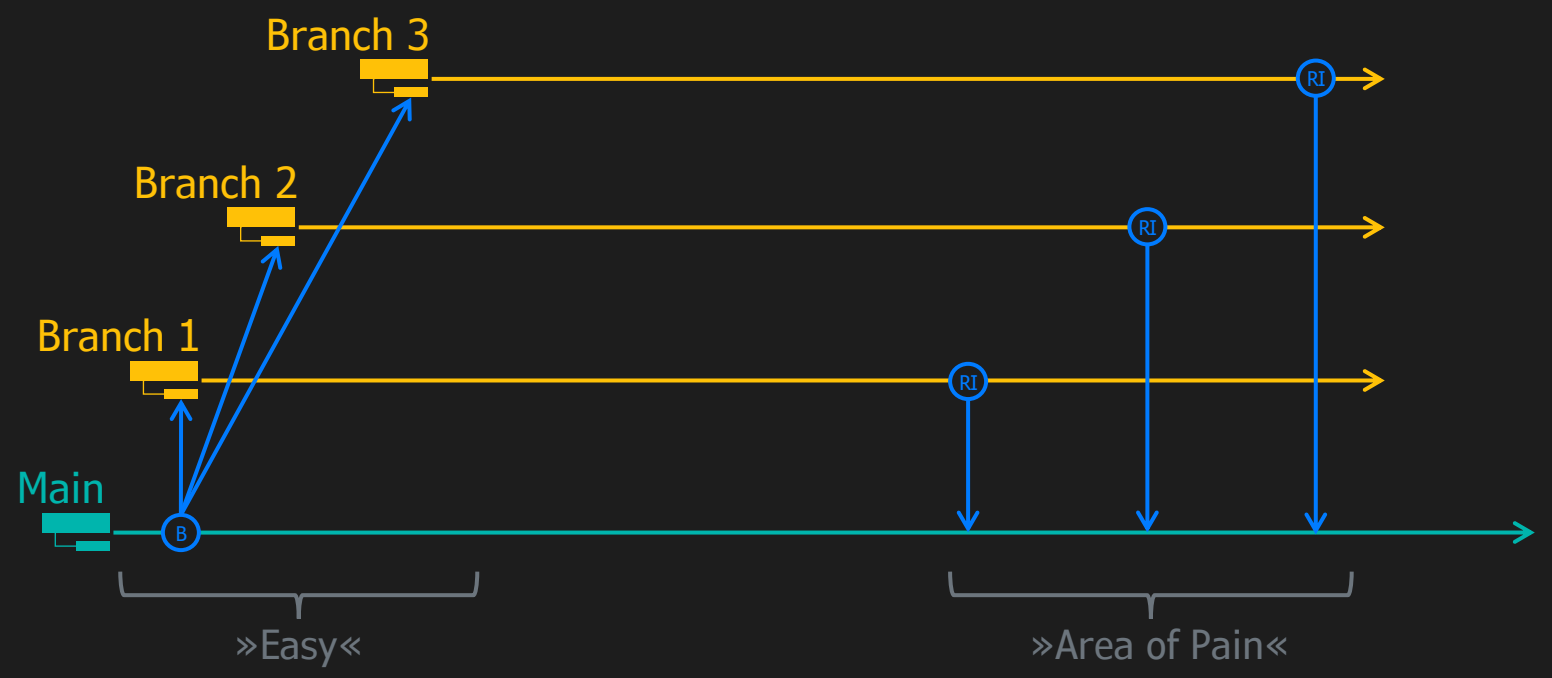
- Beispiel: Git, BitKeeper, Mercurial

Branching (1)

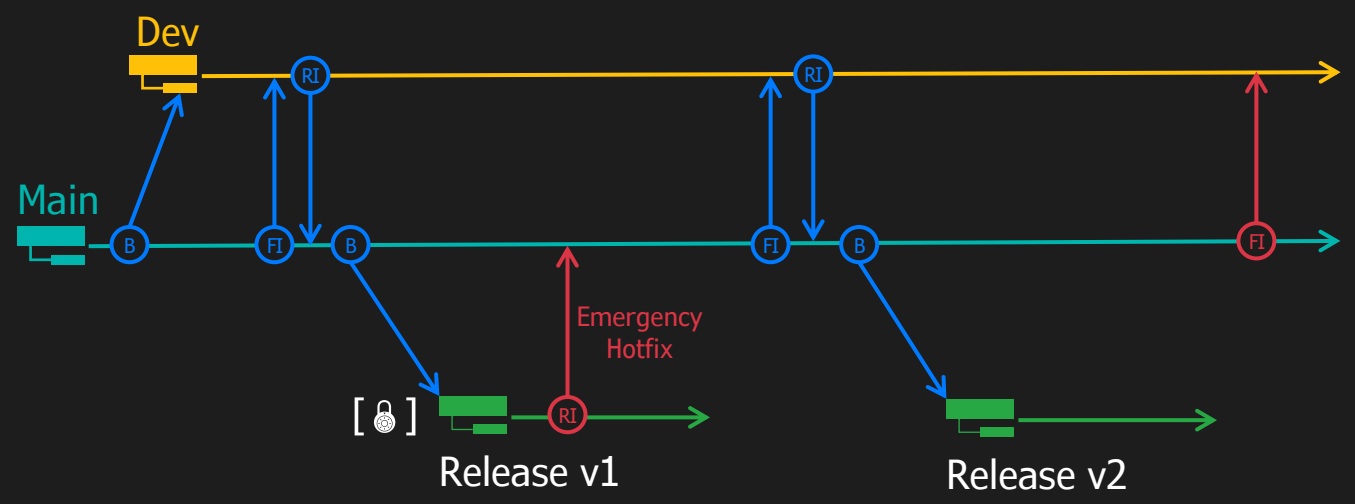
- Branching für die Arbeit in verteilten Entwicklungsteams
  - Teams arbeiten an verschiedenen Features, die weitgehend überschneidungsfrei sind, aber Abhängigkeiten untereinander besitzen (Feature-Driven Development, FDD)
  - Die Arbeiten der Teams müssen zusammengeführt werden, wobei Änderungen konsolidiert & unbeabsichtigte Auswirkungen reduziert werden müssen
  - Testende benötigen stabile Version zum Testen, während Entwickelnde ihre Arbeit fortsetzen wollen (mit zwischenzeitlich eventuell instabilen Code-Versionen)
  - Weiterentwicklung an Entwicklungsversionen, während vorhandene Produktivversionen unterstützt werden müssen, z.B. durch Service Packs, Critical Fixes oder Security Patches
- Hinweis
  - Branching erfolgt innerhalb eines Repository, siehe Git
  - Für große Softwaresysteme oder viele Teams ist auch Forking, d.h. eine (vollständige oder teilweise) Kopie des gesamten Repository eine sinnvolle Alternative

Branching (2)

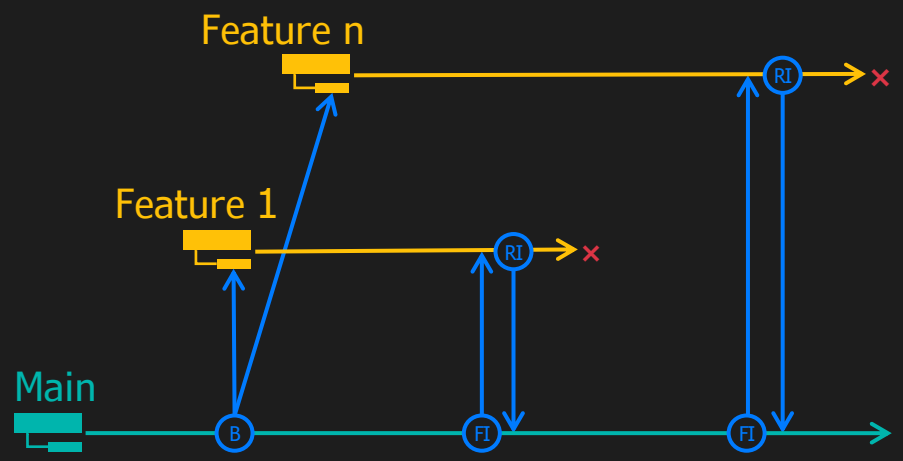
- Branches erhöhen die Komplexität der Code-Verwaltung stark
- Beim Merge können Konflikte im Quellcode entstehen, die gelöst werden müssen
- Branching ist leicht, Merge oft schmerzhaft (Merge Hell)
- Braching Strategy festlegen (Projekt oder Organisation)



Branching Strategy: Beispiele



Development and Release Isolation



Feature-Driven Development (FDD)

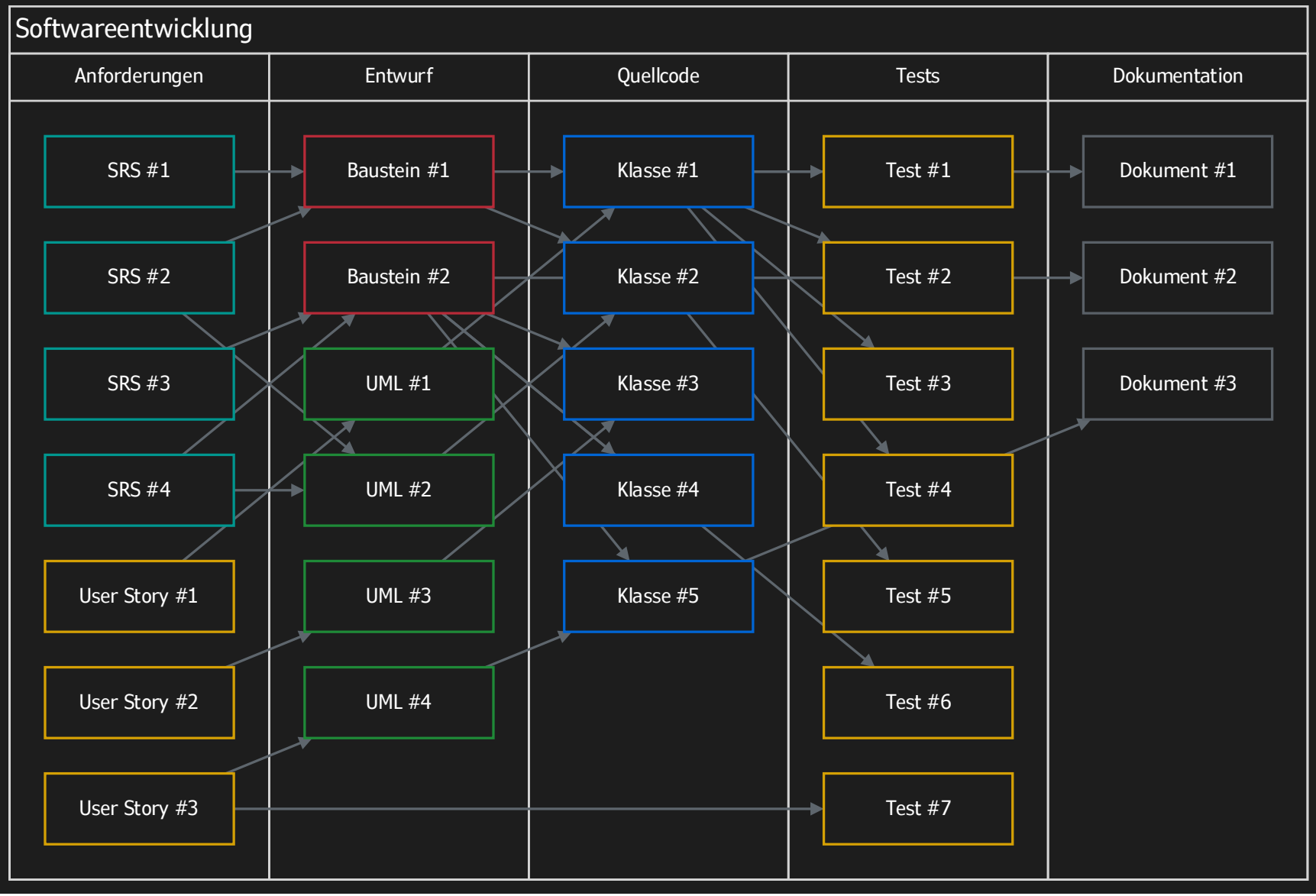
Rückverfolgbarkeit (Traceability)

Traceability – Degree to which a relationship can be established between two or more products of the development process, especially products having a predecessor-successor or master-subordinate relationship to one another.

(ISO/IEC/IEEE 24765:2017)

- Pre-Requirements Traceability
  - Woher stammt welche Anforderung? Wer hat diese formuliert? Welche Stakeholder sind betroffen? Welche Priorität haben die einzelnen Anforderungen? Wie hängen die Anforderungen zusammen?
- Post-Requirements Traceability
  - Welche Anforderungen gehören zu welchen Artefakten im Entwicklungsprozess (Modelle, Code, Testfälle, Dokumentation etc.)?

Traceability: Beispiel



Rückverfolgbarkeit: Gründe für den Einsatz

- Auswirkungsanalyse (Impact Analysis)
  - Was würde bei einer Änderung passieren? Welche Komponenten wären betroffen?
- Anforderungsüberdeckung (Requirements Coverage Analysis)
  - Welche Anforderungen wurden bei der Umsetzung erfüllt?
- Testüberdeckung (Test Coverage Analysis)
  - Welche Anforderungen wurden in welchem Umfang getestet?
- Nachvollziehbarkeit von Tests, IT-Governance-Kriterien etc.
- Beurteilung von Produktqualität und Projektfortschritt; Verständlichkeit von Fortschritts- und Abschlussberichten
- Wiederverwendung von Komponenten
- Fehlerursachensuche und Debugging